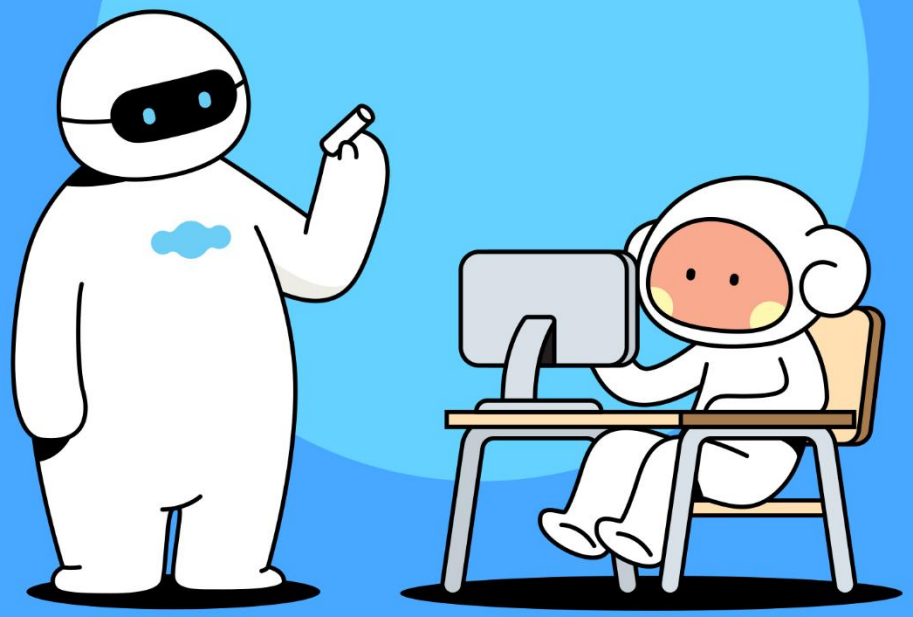
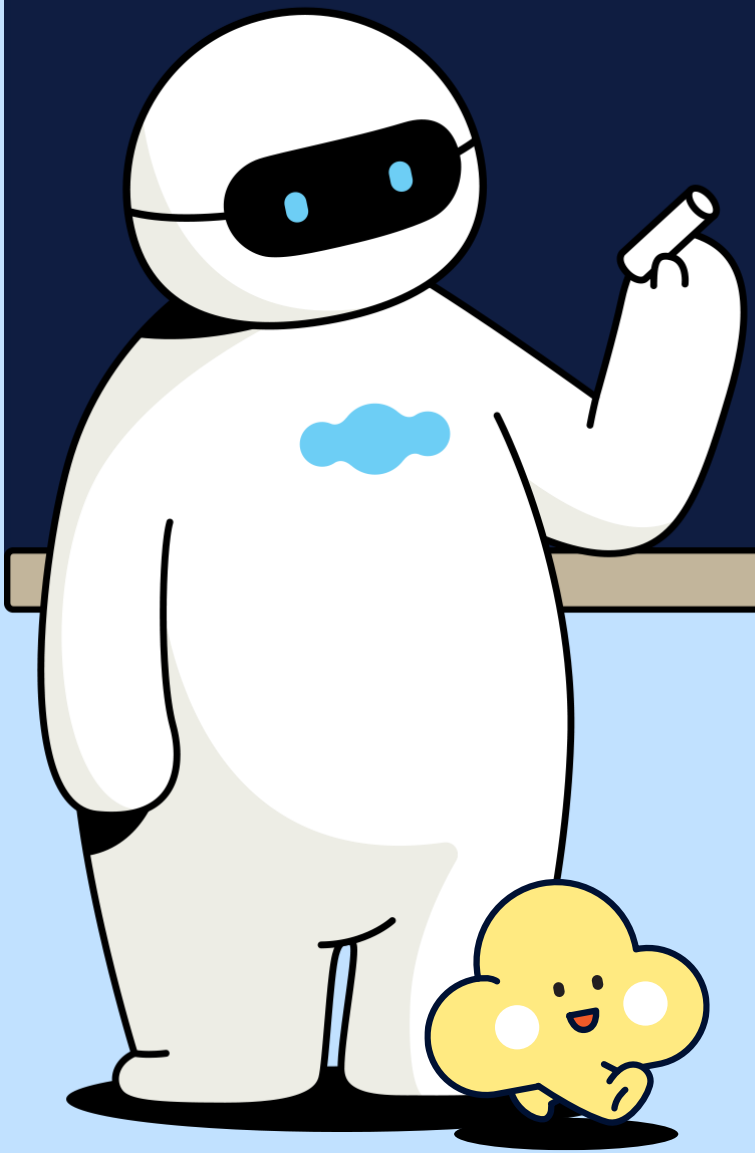
INNOGRID

EDU SQUARE

클라우드 네이티브



2026.03.17



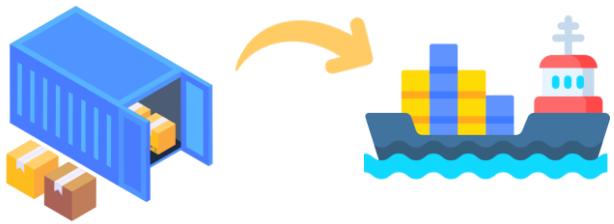
01. 컨테이너 기술 및 도구 이해

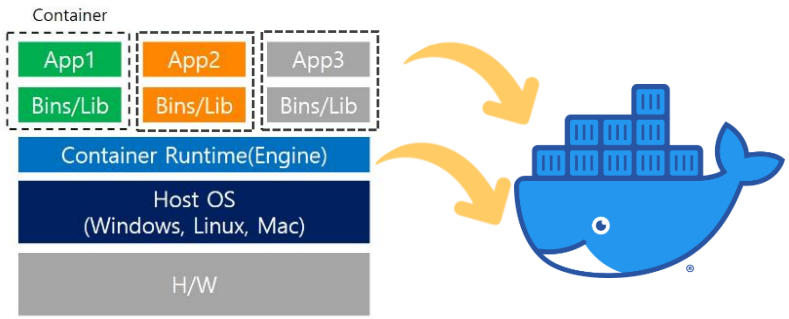
01. Container
02. Docker
03. K8S

01 Container

- **정의**

Container란





‘ 컨테이너 (Container) ’

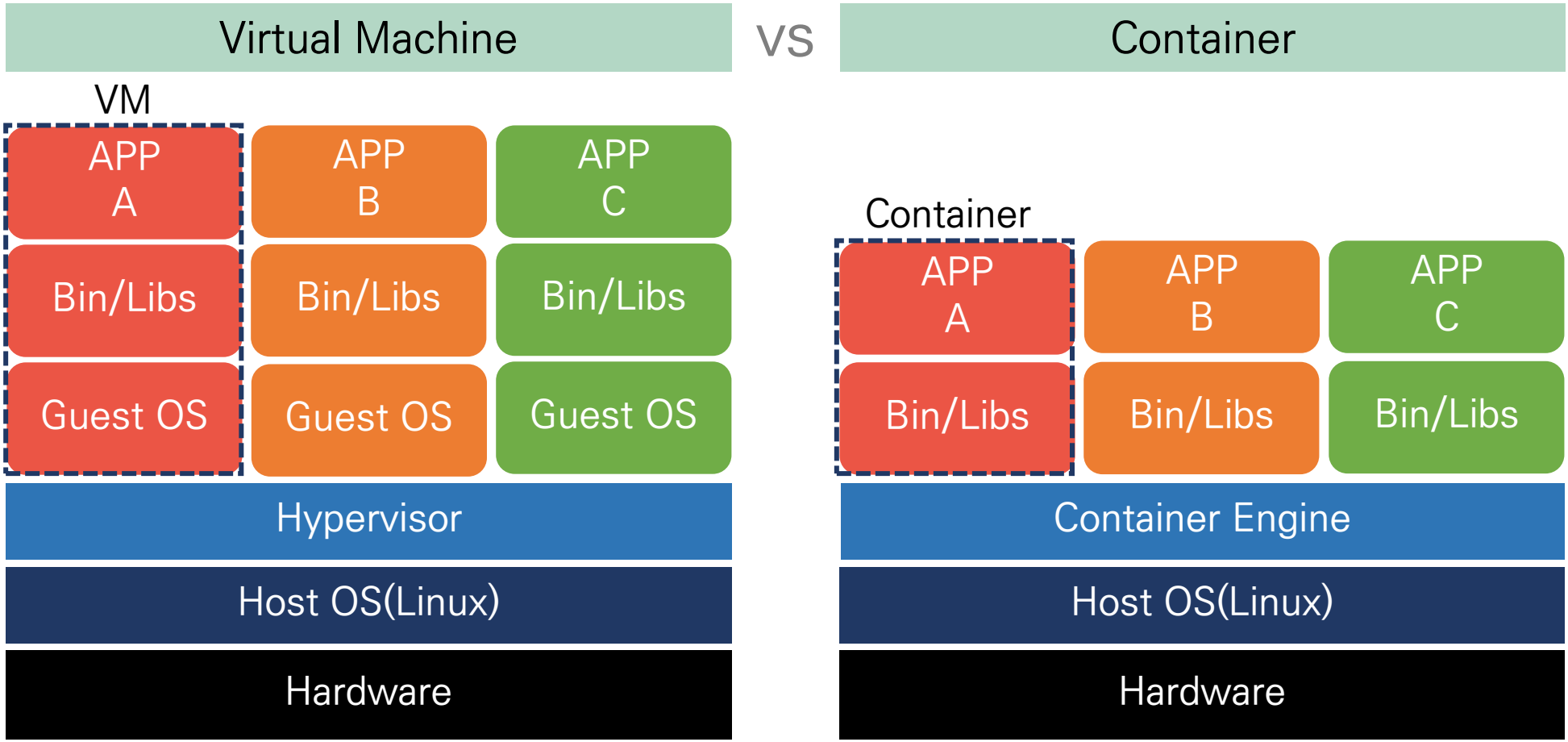
가상화된 운영체제 위에서
애플리케이션의 독립적인 실행에
필요한 파일(소스코드, 라이브러리 등)을
모은 패키지

애플리케이션의 코드를 관련 구성 파일,
라이브러리 및 앱 실행에 필요한 종속성과
함께 번들로 제공하는 SW패키지

출처 : Google Cloud 홈페이지, Azure 홈페이지

01 Container

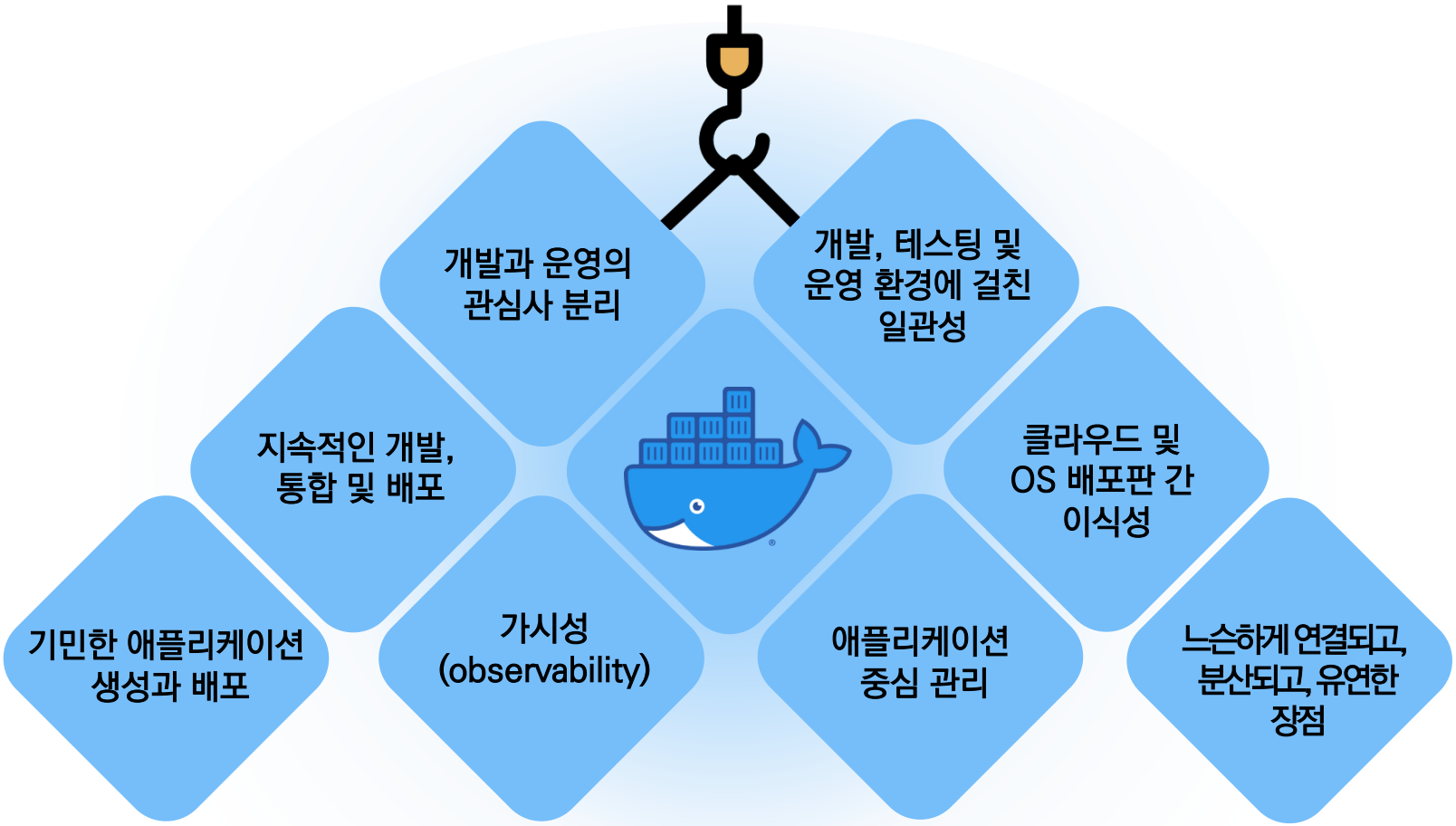
- **Hypervisor 기술과 Container 기술**



출처 : Google Cloud 홈페이지, Azure 홈페이지

01 Container

- **특징**



출처 : 쿠버네티스 홈페이지(https://kubernetes.io/ko/docs/concepts/overview/)

01 Container

- **특징**

Container 특징

- 쉽고 효율적인 컨테이너 이미지 생성으로 애플리케이션을 기민하게 생성, 배포할 수 있음
- 안정적, 주기적으로 컨테이너 이미지를 빌드, 배포할 수 있기 때문에 지속적인 개발, 통합 및 배포 가능
- 빌드/릴리즈 시점에 애플리케이션 컨테이너 이미지를 만들기 때문에 개발과 운영의 관심사 분리
- 애플리케이션의 헬스와 그 밖의 시그널을 볼 수 있어 가시성(Observability)이 높음
- Laptop, 클라우드에서 모두 동일하게 구동되므로 개발, 테스팅 및 운영 환경에 걸친 일관성 확보
- 주요 퍼블릭 클라우드와 어디에서든 구동되어 클라우드 및 OS 배포판 간 이식성이 높음
- 논리적 리소스를 사용하는 OS 중심으로 추상화 수준이 높아져 애플리케이션 중심 관리 가능
- 느슨하게 묶이고, 분산되고, 유연하며, 자유로운 마이크로 서비스 형태로 애플리케이션 배포, 관리
- 리소스가 격리되어 애플리케이션 성능을 예측할 수 있음
- 고효율 고집적의 리소스 사용량

출처 : 쿠버네티스 홈페이지(https://kubernetes.io/ko/docs/concepts/overview/)

01 Container

Container Engine & Container Runtime

Container Engine

- 컨테이너를 관리하기 위한 API나 CLI 도구를 제공하는 소프트웨어
- 컨테이너 엔진은 사용자 입력을 받고, 컨테이너 이미지를 내려 받고(pull), 컨테이너 실행 방법을 명시한 메타데이터를 만든 다음, 컨테이너 런타임에 이 정보들을 전달
- 도커 엔진(docker-ce), Redhat의 파드맨(Podman), 로켓 컴퍼니의 rkt 등

Container Runtime

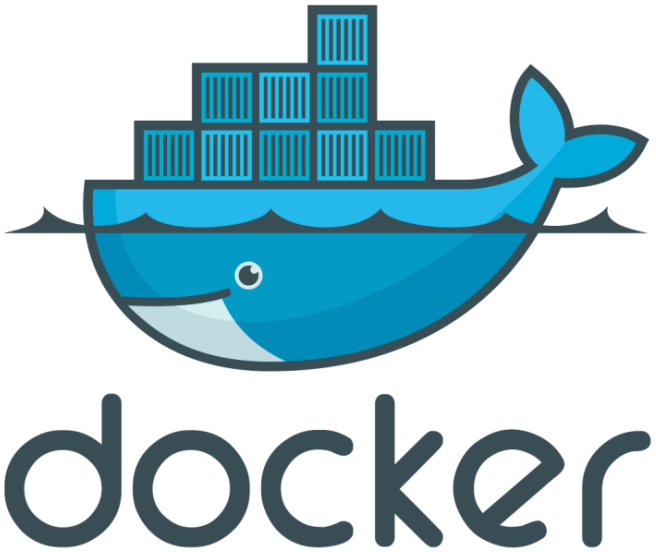
- 루트 파일시스템과 메타 데이터(spec file)를 받아 컨테이너를 실행하는 도구
- 가장 일반적으로 쓰이는 런타임은 OCI를 준수하는 runC
- 컨테이너-디 (containerd), 크라이-오 (cri-o) 등도 runC에 의존

출처: Redhat 블로그(https://developers.redhat.com/blog/2018/02/22/container-terminology-practical-introduction#namespace)

02 Docker

- **정의**

도커란



도커

애플리케이션과 실행환경을 포함한
컨테이너를 관리 및 운영하는
오픈 소스 가상화 플랫폼

출처: 도커 공식 문서(https://www.docker.com/)

02 Docker

- **특징**

도커 특징

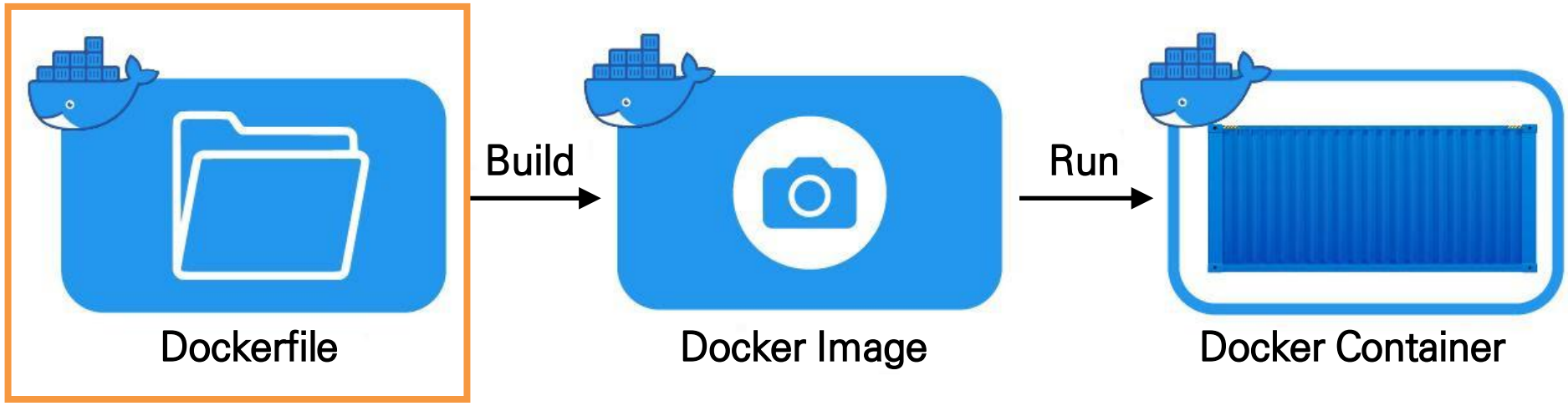
- 빠른 실행 속도
- 손 쉬운 배포
- 가벼운 용량
- 애플리케이션의 독립성
- 높은 보안성 보장
- 낮은 오버헤드
- 이미지 생성 및 공유

출처: 도커 공식 문서(https://www.docker.com/)

02 Docker

Dockerfile

도커 파일



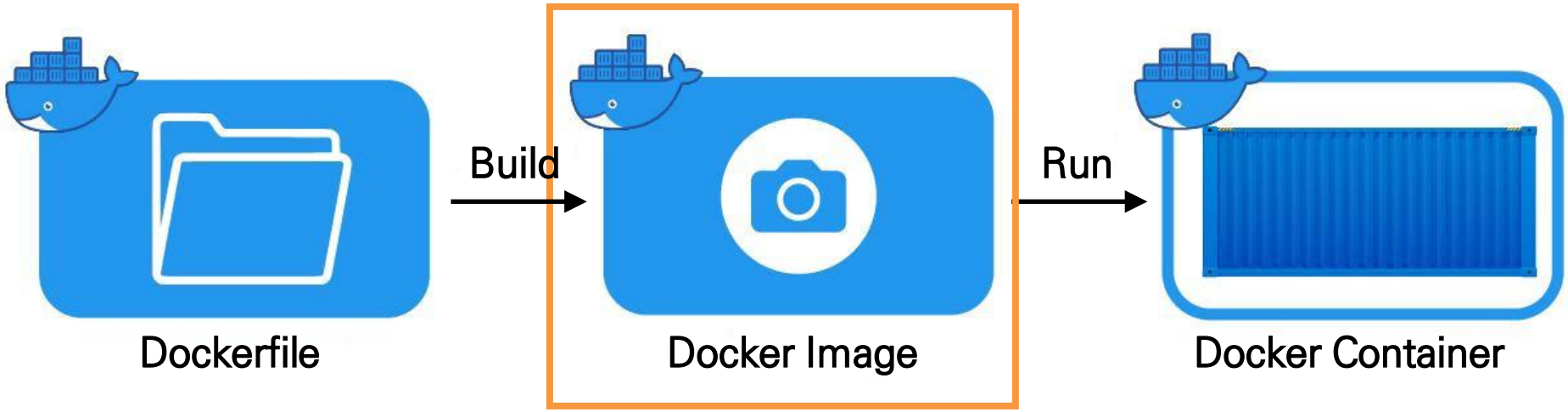
Dockerfile
도커 파일은 도커 이미지를 생성하기 위한 설정 파일이며 다양한 도커 파일 명령어를 사용하여 작성

출처 : A Beginner's Guide to Understanding and Building Docker Images (https://jfrog.com/devops-tools/article/understanding-and-building-docker-images/)

02 Docker

Docker Image

도커 이미지



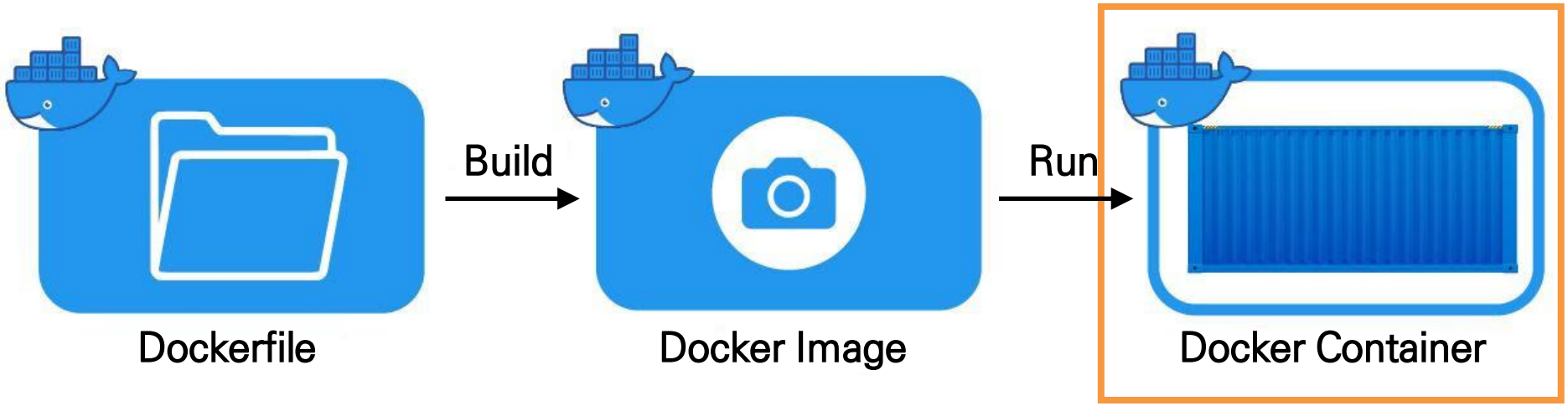
Docker Image
도커 이미지는 컨테이너 실행에 필요한 파일과 설정 값 등을 포함하고 있으며 도커 파일을 빌드하여 생성

출처 : A Beginner's Guide to Understanding and Building Docker Images (https://jfrog.com/devops-tools/article/understanding-and-building-docker-images/)

02 Docker

Docker Container

도커 컨테이너



Docker Container

도커 컨테이너는 격리된 공간에서 애플리케이션을 동작시킨 상태이며 도커 이미지를 Run하여 도커 컨테이너를 실행

출처 : A Beginner's Guide to Understanding and Building Docker Images (https://jfrog.com/devops-tools/article/understanding-and-building-docker-images/)

02 Docker

Podman 정의

**리눅스 시스템에서 컨테이너 및 컨테이너 이미지를
실행, 관리, 배포할 수 있도록 설계된 Daemonless 컨테이너 엔진**

podman

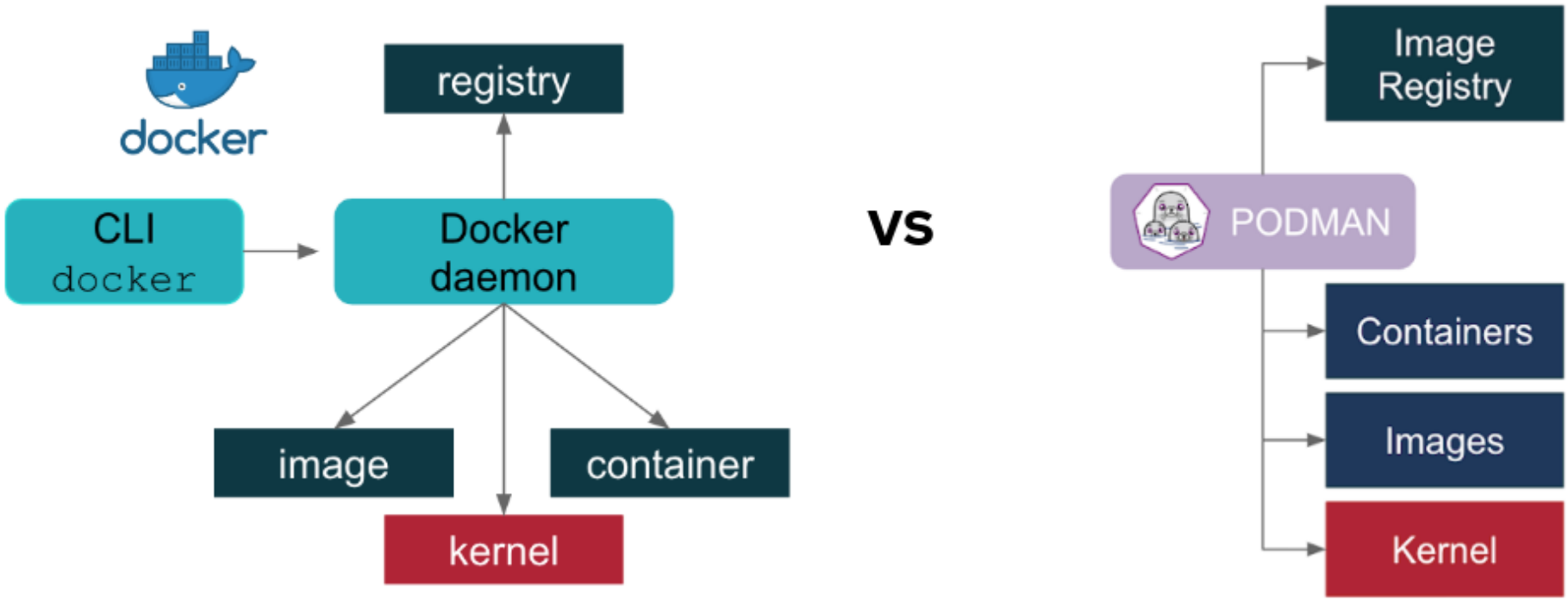
- Podman은 Pod Manager tool의 약자
- Rootless 컨테이너와 Pod를 위한 리소스를 실행하고 분리하여 더 손쉽게 액세스할 수 있는 컨테이너 환경을 만드는 동시에 보안 리스크를 줄일 수 있음
- OCI(Open Container Initiative)를 준수하며 Docker와 거의 대부분의 CLI Command가 동일
- 컨테이너를 관리하기 위해 RESTful API를 배포

* *Daemon* – 서비스 요청에 대해 응답하기 위해 오랫동안 (long-running) 실행중인 백그라운드 프로세스

출처: 파드맨 공식 문서(https://docs.podman.io/en/latest/#)

02 Docker

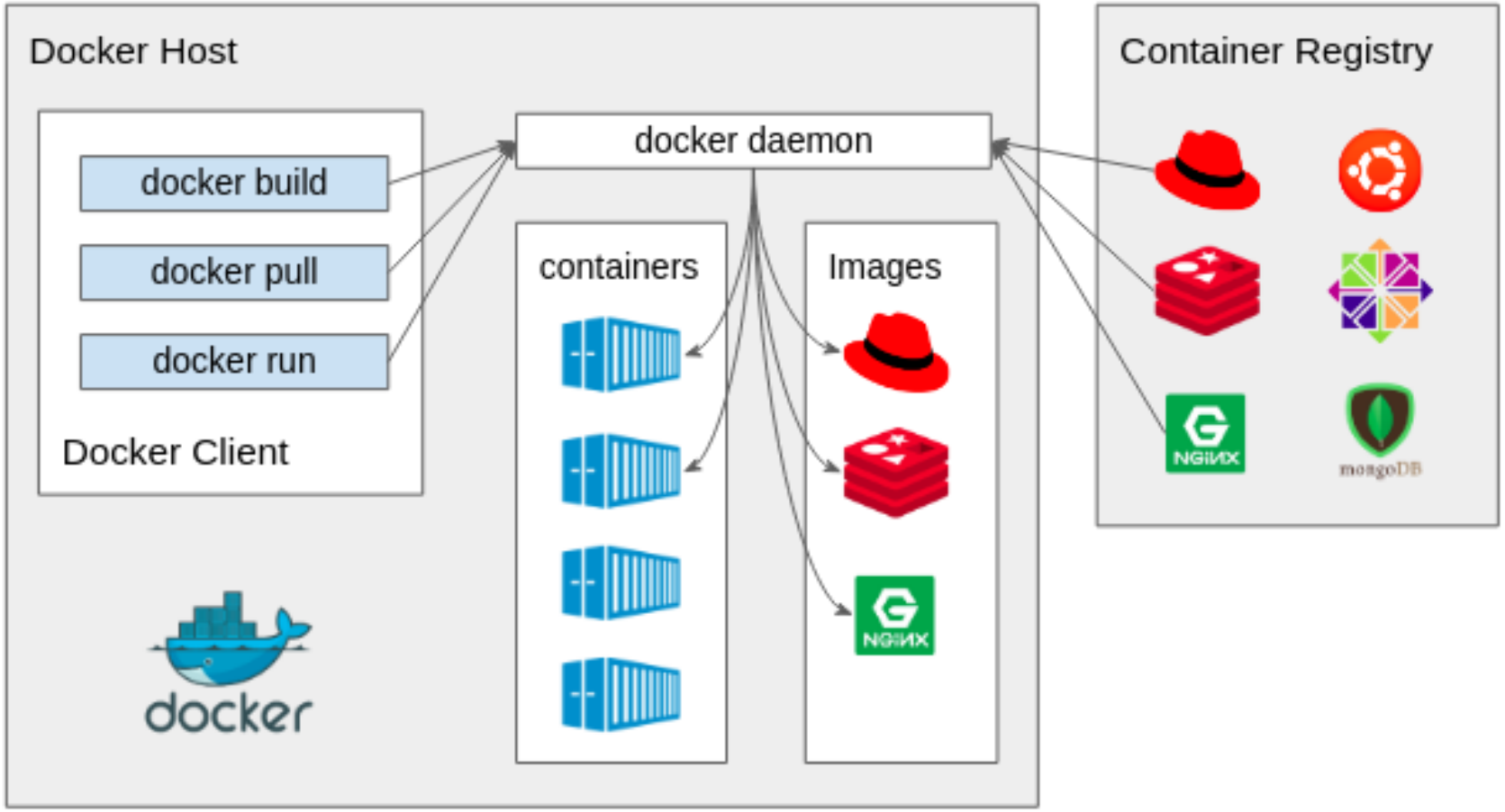
- **Docker 와 Podman 차이**



출처 : Open Source Summit Japan 2019 발표자료(https://www.slideshare.net/DanielOh20/podman-buildah-and-quarkus-the-latest-in-linux-containers-technologies)

02 Docker

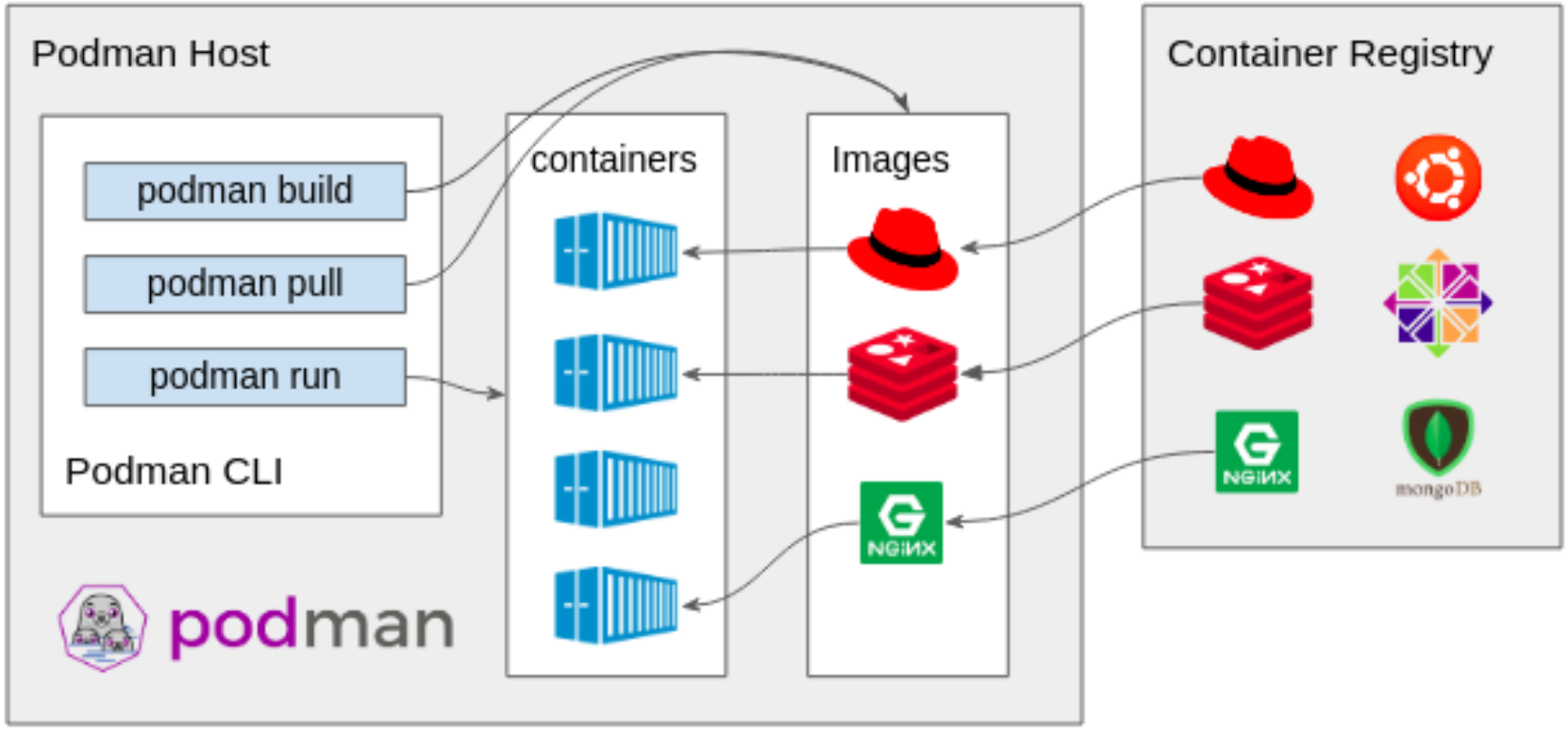
- **Docker 와 Podman 차이**



출처 : Open Source Summit Japan 2019 발표자료(https://www.slideshare.net/DanielOh20/podman-buildah-and-quarkus-the-latest-in-linux-containers-technologies)

02 Docker

- **Docker 와 Podman 차이**



출처 : Open Source Summit Japan 2019 발표자료(https://www.slideshare.net/DanielOh20/podman-buildah-and-quarkus-the-latest-in-linux-containers-technologies)

02 Docker

Dockerfile 작성

도커 파일 예시 (1/3)

```
FROM tomcat:9-jre8-alpine
WORKDIR /usr/local/tomcat
COPY server.xml ./conf
RUN rm -rf ./webapps/*
ARG JAR_FILE=*.war
COPY ${JAR_FILE} ./webapps/edu-msa-board-1.0.0.war

EXPOSE 28082
```

- **FROM**
  컨테이너 기반이 되는 베이스 이미지
- **WORKDIR**
  명령을 실행할 작업 디렉토리 설정
- **COPY**
  컨테이너 빌드 시 컨테이너로 파일 복사

출처: 도커 공식 문서(https://www.docker.com/)

02 Docker

Dockerfile 작성

도커 파일 예시 (2/3)

```
FROM tomcat:9-jre8-alpine
WORKDIR /usr/local/tomcat
COPY server.xml ./conf
RUN rm -rf ./webapps/*
ARG JAR_FILE=*.war
COPY ${JAR_FILE} ./webapps/edu-msa-board-1.0.0.war

EXPOSE 28082
```

- RUN
  베이스 이미지에서 실행할 명령
- ARG
  파일에 사용할 변수 값 설정
- EXPOSE
  외부에 노출시킬 포트 번호

출처: 도커 공식 문서(https://www.docker.com/)

02 Docker

- **Dockerfile 작성**

도커 파일 예시 (3/3)

```
FROM openjdk:8-jdk-alpine
ARG JAR_FILE=edu-msa-zuul-1.0.0.war
COPY ${JAR_FILE} edu-msa-zuul-1.0.0.war
COPY application.yml /application.yml
ENTRYPOINT ["java","-jar","-Dspring.config.location=application.yml","/edu-msa-zuul-1.0.0.war"]
```

- ENTRYPOINT
  컨테이너 실행 시 수행되는 명령어를 지정(프로파일 값 변경)

출처: 도커 공식 문서(https://www.docker.com/)

02 Docker

Docker CLI

Docker CLI 개요

특징

Docker 명령은 Docker-CLI와 유사한 CLI를 사용하며 대부분 추가 권한 없이
일반 사용자로 실행 가능

기본 형태

$ docker [*options*] command

02 Docker

Docker CLI

$ docker version

명령 의미

Docker 버전 정보를 표시

기본 형태

docker version [*options*]

사용 예시

```
$ docker version
Client: Docker Engine - Community
 Version:           26.1.3
 API version:       1.45
 Go version:        go1.21.10
 Git commit:        b72abbb
 Built:             Thu May 16 08:33:29 2024
 OS/Arch:           linux/amd64
 Context:           default
```

$ docker login(또는 logout)

명령 의미

컨테이너 레지스트리에 로그인/로그아웃

기본 형태

docker login [*options*] [*registry*]

사용 예시

```
$ docker login docker.io
Username: ubuntu
Password: Login Succeeded!
```

02 Docker

Docker CLI

$ docker build

명령 의미

컨테이너 파일을 사용하여 컨테이너 이미지 빌드

기본 형태

docker build [*options*] [*context*]

사용 예시

```
$ docker build .

$ docker build -f [Containerfile.simple] .

$ docker build --tag [imageName]:latest .
```

$ docker tag

명령 의미

로컬 이미지에 새로운 이름 추가

기본 형태

docker tag *image*[*:tag*] [*target-name*[*:tag*]…] [*options*]

사용 예시

```
$ docker tag 0e3bbc2 fedora:latest

$ docker tag httpd myregistryhost:5000/fedora/httpd:v2

$ docker tag edu-msa-board ${도커허브 ID}/edu-msa-board
```

02 Docker

Docker CLI

$ docker pull

명령 의미

레지스트리에서 이미지 가져오기

기본 형태

docker pull [*options*] source [*source*…]

사용 예시

```
# Pull from a container registry
$ docker pull quay.io/username/myimage

# Pull from a container registry with short-name resolution
$ docker pull fedora
```

$ docker push

명령 의미

이미지, 매니페스트 목록 또는 이미지 인덱스를 local 스토리지에서 다른 곳으로 보내기

기본 형태

docker push [*options*] image [*destination*]

사용 예시

```
# Push to a container registry
$ docker push quay.io/docker/stable

# Push to a container registry via the docker transport
$ docker push docker://quay.io/docker/stable

$ docker push ${도커허브 ID}/edu-msa-board
```

02 Docker

Docker CLI

$ docker run

명령 의미

컨테이너 이미지로 새 컨테이너 실행

기본 형태

docker run [*options*] image [*command* [*arg* …]]

사용 예시

```
$ docker run --read-only -i -t fedora /bin/bash
$ docker run -a stdin -a stdout -i -t fedora /bin/bash
$ docker run -it --ipc=host shm ipcs -m
```

$ docker exec

명령 의미

실행 중인 컨테이너에서 명령 실행

기본 형태

docker exec [*options*] container [*command* [*arg* …]]

사용 예시

```
$ docker exec -it [컨테이너 이름] bash
$ docker exec -it ctrID ls
$ docker exec -it -w /tmp myCtr pwd
$ docker exec --user root ctrID ls
```

02 Docker

Docker CLI

$ docker stop

명령 의미

실행 중인 컨테이너 중지

기본 형태

docker stop [*options*] *container* ···

사용 예시

```
$ docker stop [컨테이너 이름 / 컨테이너 ID]
$ docker stop --cidfile [파일 경로]
$ docker stop -a
$ docker stop --latest
```

$ docker kill

명령 의미

실행 중인 컨테이너 즉시 종료

기본 형태

docker kill [*options*] [*container* ···]

사용 예시

```
$ docker kill [컨테이너 이름 / 컨테이너 ID]
$ docker kill --cidfile [파일 경로]
$ docker kill --signal KILL -a
```

02 Docker

Docker CLI

$ docker ps

명령 의미

컨테이너에 대한 정보 출력

기본 형태

docker ps [*options*]

사용 예시

```
$ docker ps -a
$ docker ps -a -s
$ docker ps --ns -a
```

$ docker rm

명령 의미

컨테이너 삭제

기본 형태

docker rm [*options*] *container*

사용 예시

```
$ docker rm [컨테이너 이름]
$ docker rm -f [컨테이너 ID]
$ docker rm -f -a
$ docker rm -f --latest
```

03 Kubernetes

- **정의**

쿠버네티스란

kubernetes

컨테이너화된 애플리케이션을
자동으로 배포, 스케일링 및 관리해주는
오픈소스 시스템

컨테이너화된 워크로드와 서비스를
관리하기 위한
이식성이 있고, 확장가능한 오픈소스 플랫폼

출처 : 쿠버네티스 홈페이지

03 Kubernetes

- **Kubernetes 특징**

쿠버네티스 특징

kubernetes

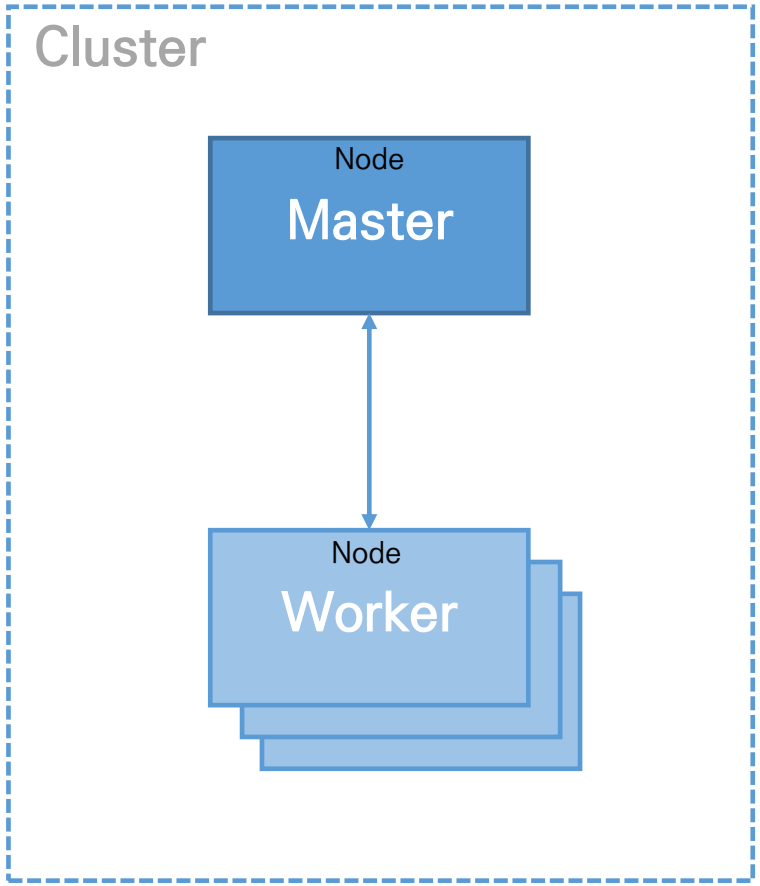
- 서비스 디스커버리와 로드 밸런싱
- 스토리지 오케스트레이션
- 자동화된 롤아웃과 롤백
- 자동화된 빈 패킹(bin packing)
- 자동화된 복구(self-healing)
- 시크릿과 구성 관리

출처 : 쿠버네티스 홈페이지

03 Kubernetes

Kubernetes Cluster 구성

클러스터 구성요소

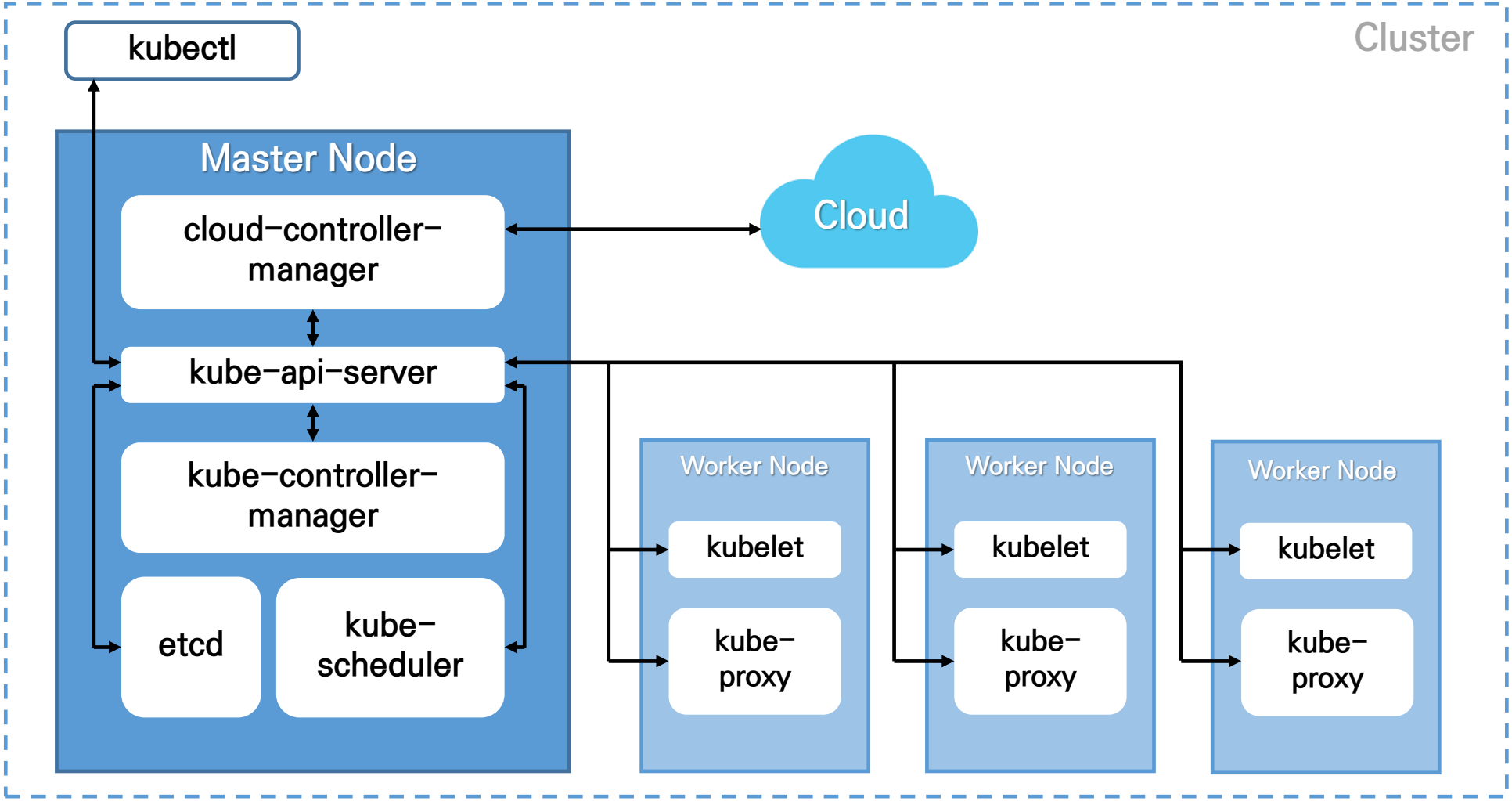


- **Cluster**
  컨테이너화된 애플리케이션을 실행하기 위한 일련의 노드 머신의 집합
- **Master**
  클러스터 전체를 컨트롤하며 내부에 있는 모든 노드를 관리하는 가상 머신
- **Worker**
  마스터에 의해 명령을 받고 실제 컨테이너들이 생성되고 일을 하는 가상 머신

출처 : 쿠버네티스 홈페이지

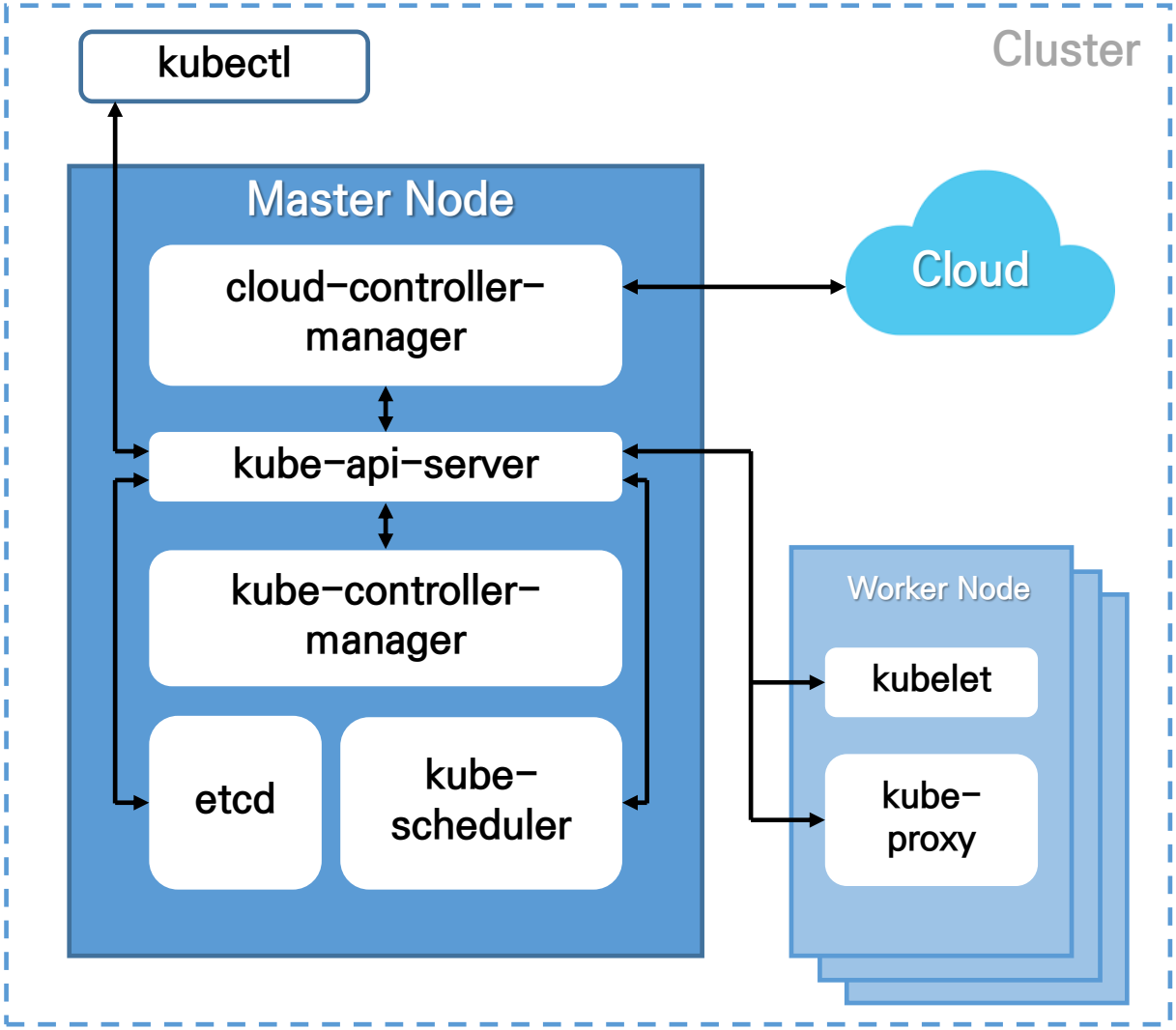
03 Kubernetes

- **Kubernetes Cluster 구성**



03 Kubernetes

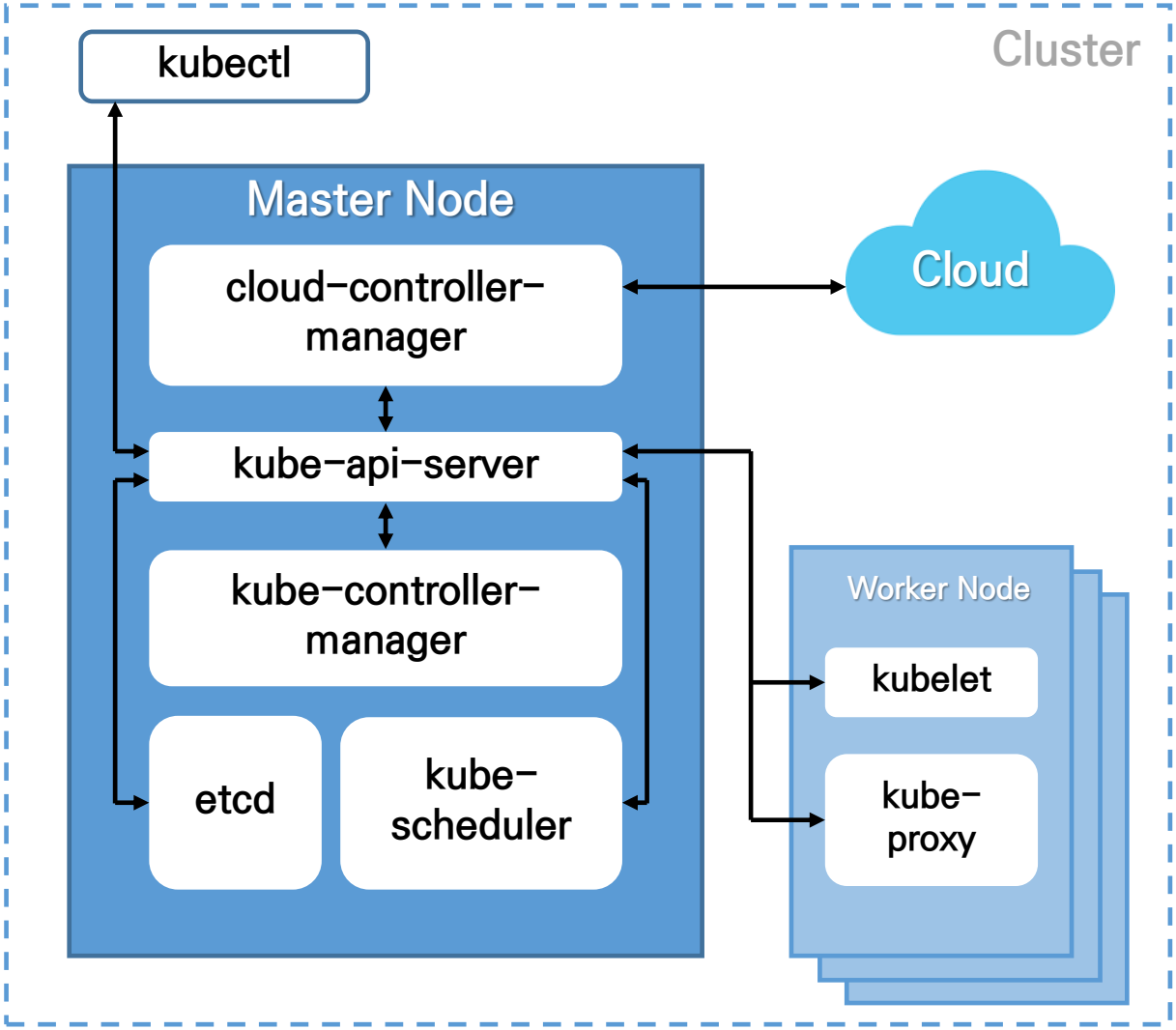
Kubernetes Cluster 구성



- **kube-api-server**
  쿠버네티스의 모든 통신은 kube-api-server를 통해 통신
- **kube-controller-manager**
  파드들을 관리하는 각각의 컨트롤러를 제어하는 역할
- **kube-scheduler**
  리소스를 자원 할당이 가능한 노드에 할당하는 역할
- **etcd**
  클러스터 내의 모든 세부적인 데이터를 저장하는 키-값 저장소

03 Kubernetes

- **Kubernetes Cluster 구성**

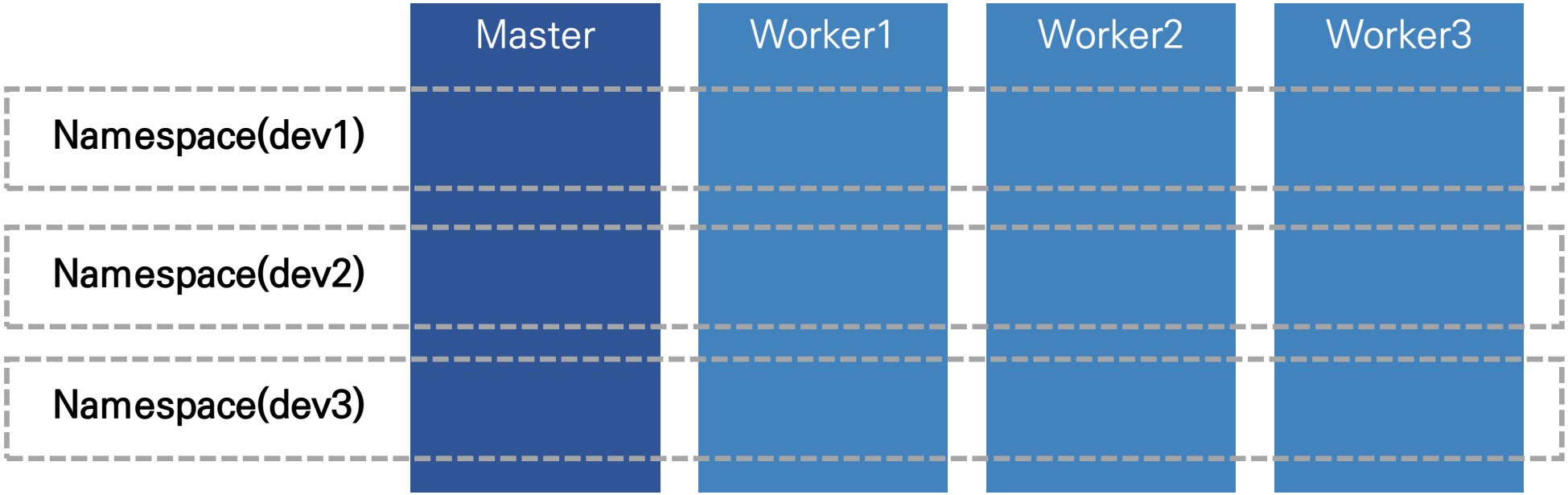


- cloud-controll-manager
  컨트롤러들을 클라우드 서비스와 연결하여 관리
- kubelet
  모든 노드에서 실행되며 컨테이너 실행 및 지속적인 헬스체크를 통해 마스터의 kube-api-server와 통신을 하는 에이전트
- kube-proxy
  클러스터 내부의 가상 네트워크를 설정하여 관리

03 Kubernetes

Namespace

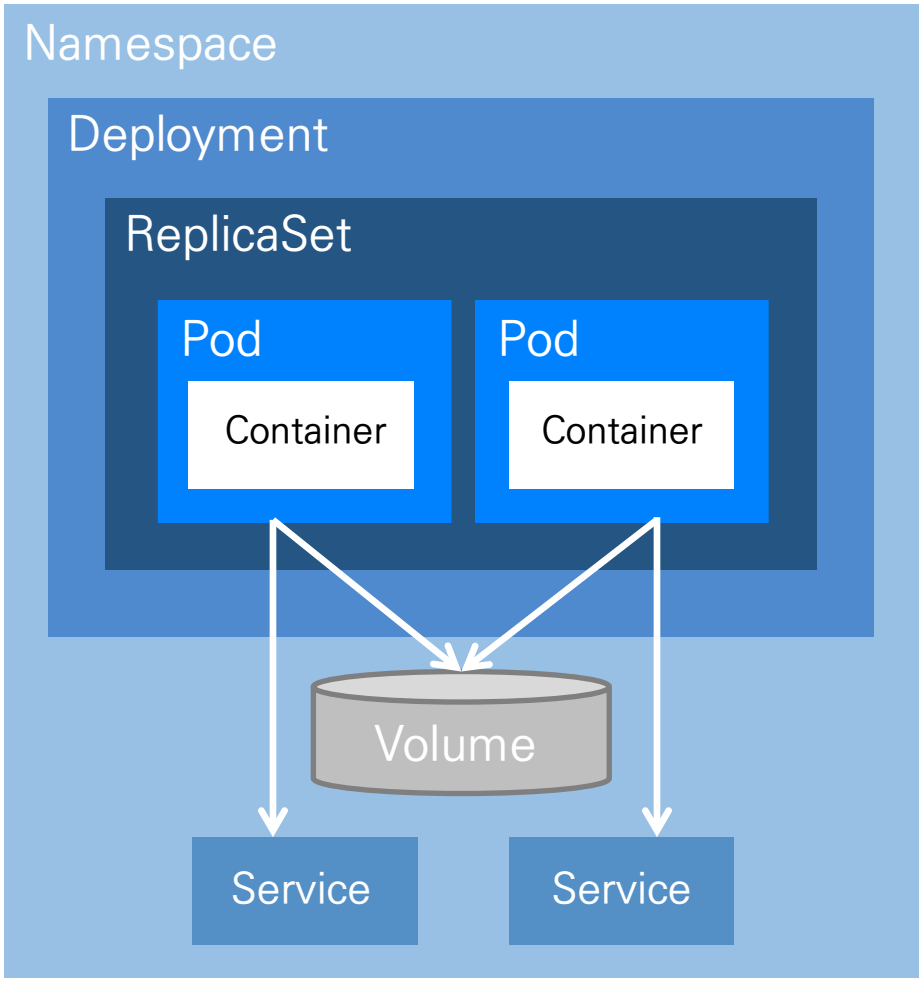
네임스페이스는 클러스터를 논리적으로 분리하여 사용하는 것을 의미



- 네임스페이스는 물리적인 마스터/워커 노드들 논리적인 단위로 분리한 리소스이며 각 사용자는 권한이 부여된 네임스페이스만 접근 가능
  예) dev1 네임스페이스에 대한 권한을 받은 사용자는 dev1만 접근 가능

03 Kubernetes

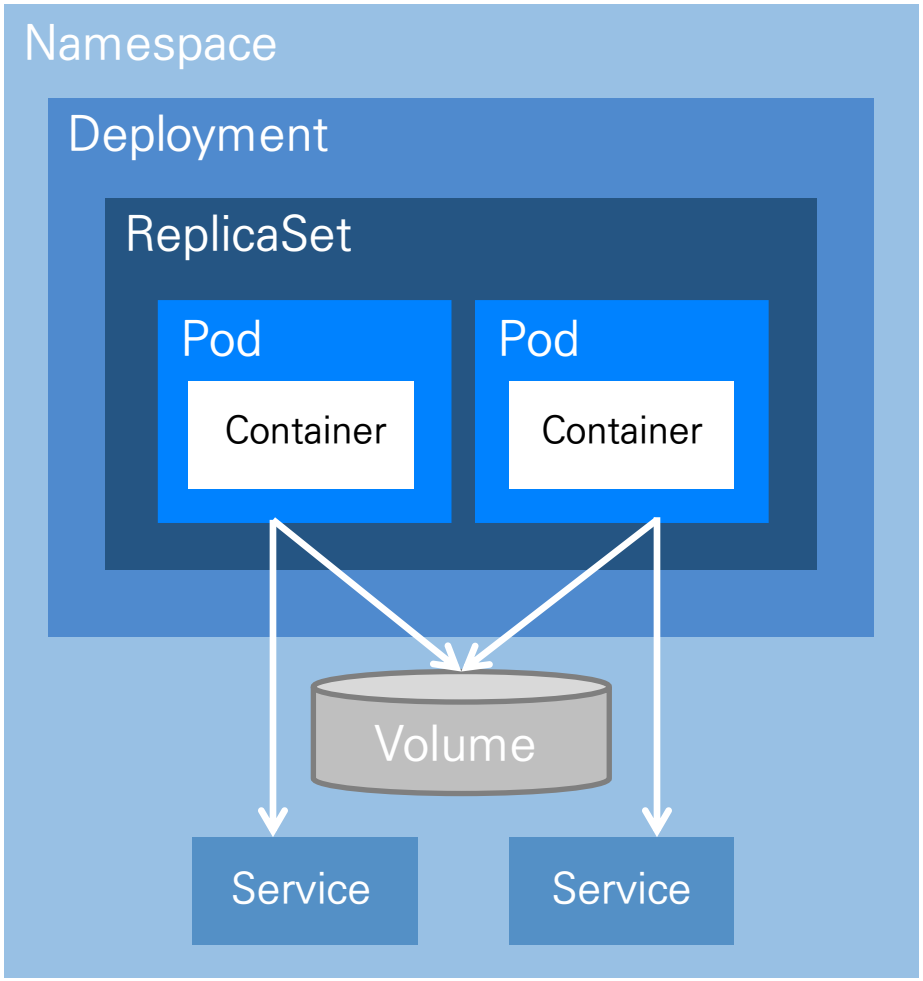
Resource



- **Namespace**
  동일한 물리 클러스터 하나를 여러 개의 논리적인 단위로 나누는 리소스
- **Deployment**
  레플리카셋을 관리하면서 실행시켜야 할 파드의 배포 및 관리를 하는 리소스
- **ReplicaSet**
  파드의 개수를 유지하고 관리하는 리소스
- **Pod**
  실제로 컨테이너가 파드에서 실행되며 컨테이너를 돌리는 최소한의 단위

03 Kubernetes

Resource

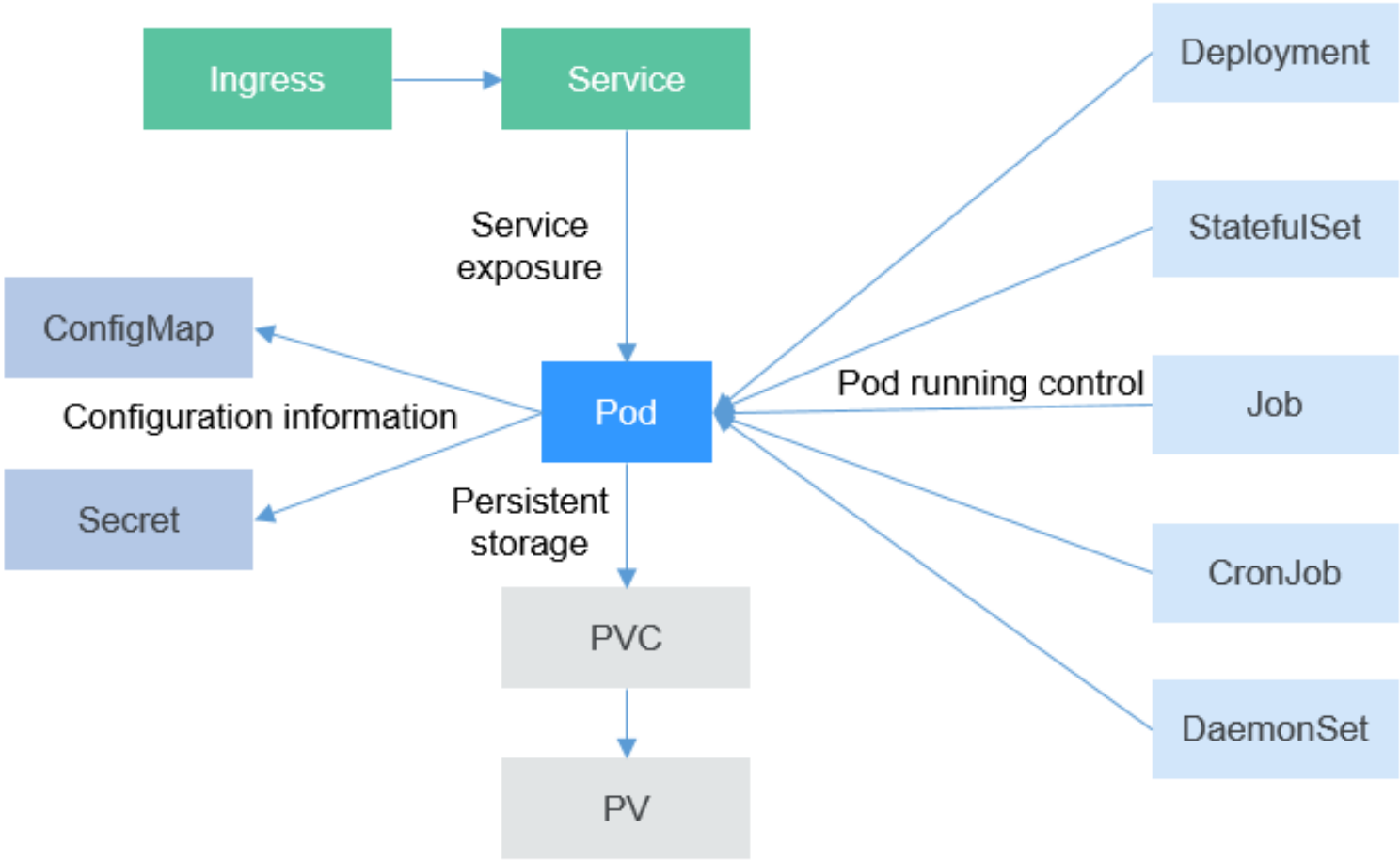


- **Service**
  클러스터 외부에서 클러스터 내부 파드에 접근 할 수 있도록 IP를 제공하는 리소스

- **Volume**
  파드의 일부분으로 동일한 파드 내의 컨테이너끼리는 볼륨 공유가 가능하며 컨테이너의 데이터를 보관하여 유지시키는 역할

03 Kubernetes

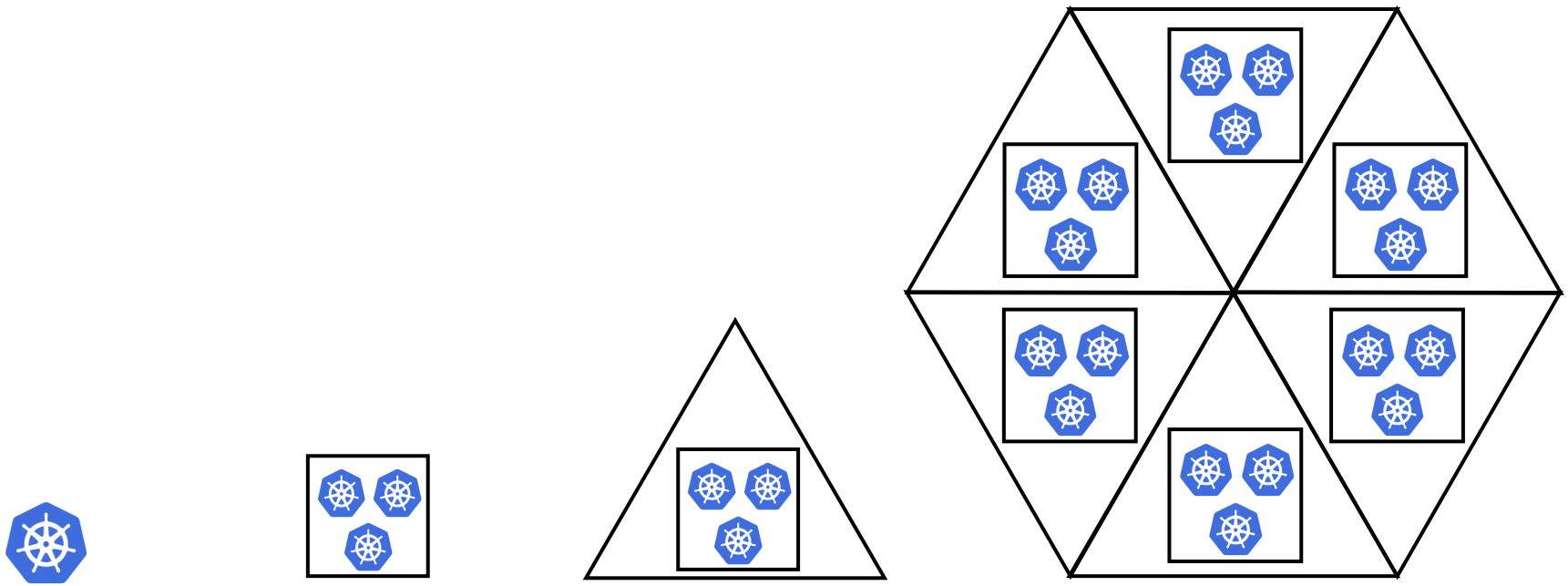
- **Resource**



03 Kubernetes

- Resource

리소스 소개

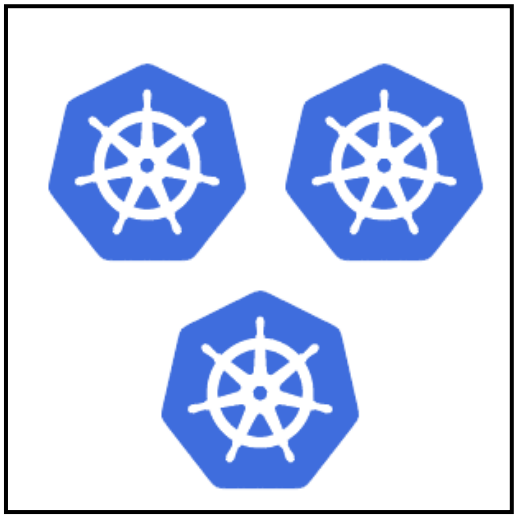


Container Pod ReplicaSet Deployment

03 Kubernetes

- **Pod**

파드 특징



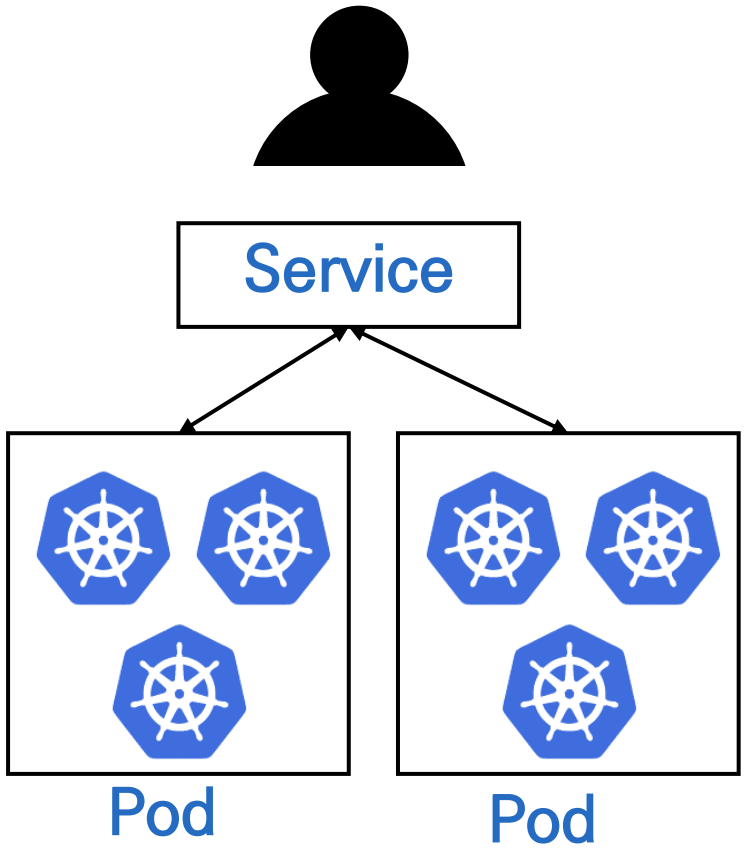
Pod

- 컨테이너가 직접 실행되는 장소
- 컨테이너를 돌리는 최소한의 단위
- 하나 이상의 컨테이너 그룹
- 컨테이너를 직접 관리하지 않고 파드 단위로 관리

03 Kubernetes

- Service

서비스 특징



- 클러스터 외부에서 클러스터 내부 파드에 접근
- 고정된 IP주소 할당
- 파드 간의 로드 밸런싱 지원
- 하나 또는 여러 개의 포트 지원

03 Kubernetes

- Ingress

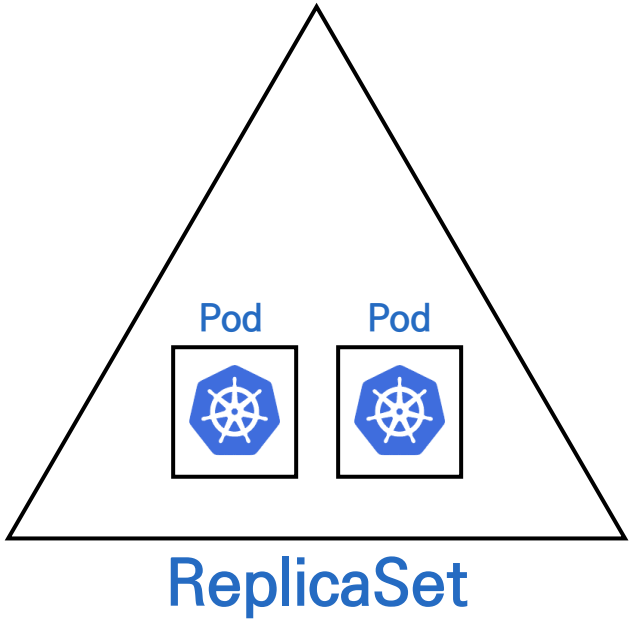
인그레스 특징

- 클러스터 외부로부터 클러스터 내부로 들어오는 네트워크 트래픽을 로드밸런싱
- L7 로드밸런싱 기능을 수행
- Service에 외부 URL을 제공
- 클러스터로 접근하는 URL 별로 다른 서비스에 트래픽을 분산
- TLS/SSL 인증서 처리
- 도메인 기반의 Virtual hosting을 지정

03 Kubernetes

ReplicaSet

레플리카셋 특징

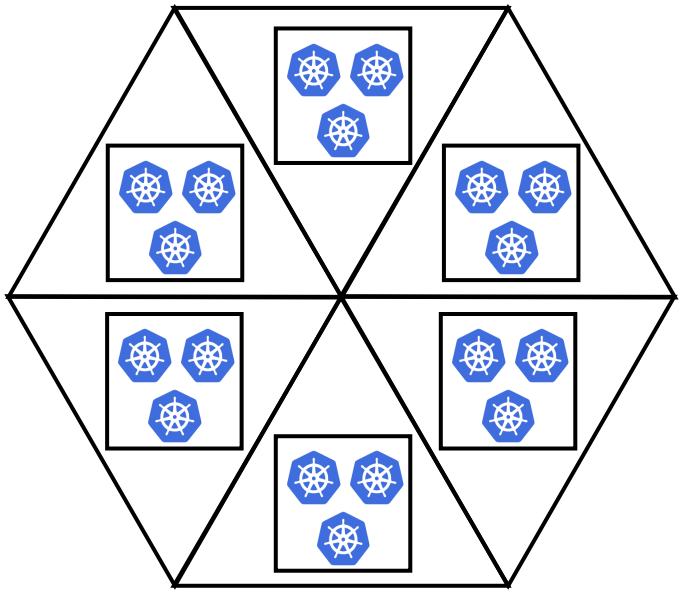


- 파드의 개수 유지 및 관리
- 실행되는 파드를 안정적으로 유지
- 파드 개수에 대한 가용성 보장

03 Kubernetes

- Deployment

디플로이먼트 특징



Deployment

- 상태가 없는 앱을 배포할 때 사용되는 가장 기본적인 컨트롤러
- 레플리카셋을 관리
- 앱의 배포를 세밀하게 관리
- 파드 개수 유지
- 배포 시 롤링 업데이트

03 Kubernetes

- StatefulSet

스테이트풀셋 특징

- 동일한 컨테이너 스펙을 가진 파드들을 관리하며 각 파드들은 각각의 식별자를 가지기 때문에 독자성을 유지
- 파드들의 순서 및 고유성을 보장
- 식별자를 가지고 있어 재스케줄링 간에도 지속적인 유지
- 내부 파드마다 고유한 pvc를 갖도록 설정이 가능하여, 스케일 확장시 PV를 유지 가능

03 Kubernetes

- **Deployment vs. StatefulSet**

Deployment	StatefulSet
Stateless 방식의 Pod 배포 상태가 없는 애플리케이션 배포	Stateful한 방식의 Pod 관리 상태가 있는 애플리케이션 배포
Service를 통한 외부 노출	Headless Service를 통한 외부 노출
Service 요청시 랜덤한 Pod 선택	요청시 원하는 Pod 선택 가능 (단, Service 요청 불가능)
ReplicaSet을 가지고 있으며 rollback 가능	ReplicaSet이 없으며 rollback 불가능
모든 Pod가 1개의 PVC에 모두 연결	Pod 마다 각각의 고유한 PVC를 생성하여 고유한 PV를 가짐

03 Kubernetes

- Job

잡 특징

Job

- 여러 파드를 지정하여 지정된 파드를 성공적으로 실행하도록 하는 설정
- 배치작업, 임시작업 등 처음 한번 실행하고 종료되는 작업에 사용
- 파드의 성공적인 완료를 보장
- 단일잡, 다중잡, 병렬잡

03 Kubernetes

- CronJob

크론잡 특징

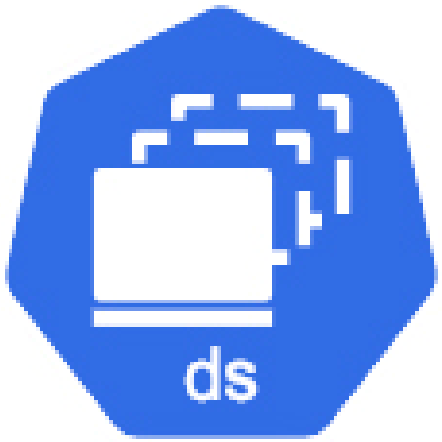
CronJob

- JOB을 실행하는데 스케줄러 역할
- 데이터 백업, 이메일 송신 등 정해진 시간에 맞춰 반복적으로 JOB을 실행해야 할 때 사용
- 리눅스의 crontab과 비슷

03 Kubernetes

- DaemonSet

데몬셋 특징



DaemonSet

- 클러스터 전체에 특정 파드를 실행할 때 사용하거나 특정 노드 또는 모든 노드에 항상 실행되어야 할 특정 파드들을 관리
- 전체 노드에서 Pod가 한 개씩 실행되도록 보장
- 로그 수집기, 모니터링 에이전트와 같이 모든 노드에 항상 실행시켜야 하는 특정 파드를 관리해야 할 때 사용

03 Kubernetes

Persistent Volume

퍼시스턴트 볼륨 특징

Persistent Volume

- 특정 파드와 상관없이 별도의 생명주기를 가지는 독립적인 볼륨
- 프로비저닝은 정적 방법, 동적 방법 두 가지 방법으로 생성
- PVC로부터 요청이 들어오면 요청에 맞는 PV 할당
- PV와 PVC 매핑은 1:1 관계로 바인딩

03 Kubernetes

- Persistent Volume Claim

퍼시스턴트 볼륨 클레임 특징

Persistent Volume Claim

- 사용자가 PV에 하는 요청
- 파드와 PV를 연결
- 요청 시 스토리지 용량, 접근방법, 셀럭터 등 설정 가능
- PV와 PVC 매핑은 1:1 관계로 바인딩

03 Kubernetes

- StorageClass

스토리지 클래스 특징

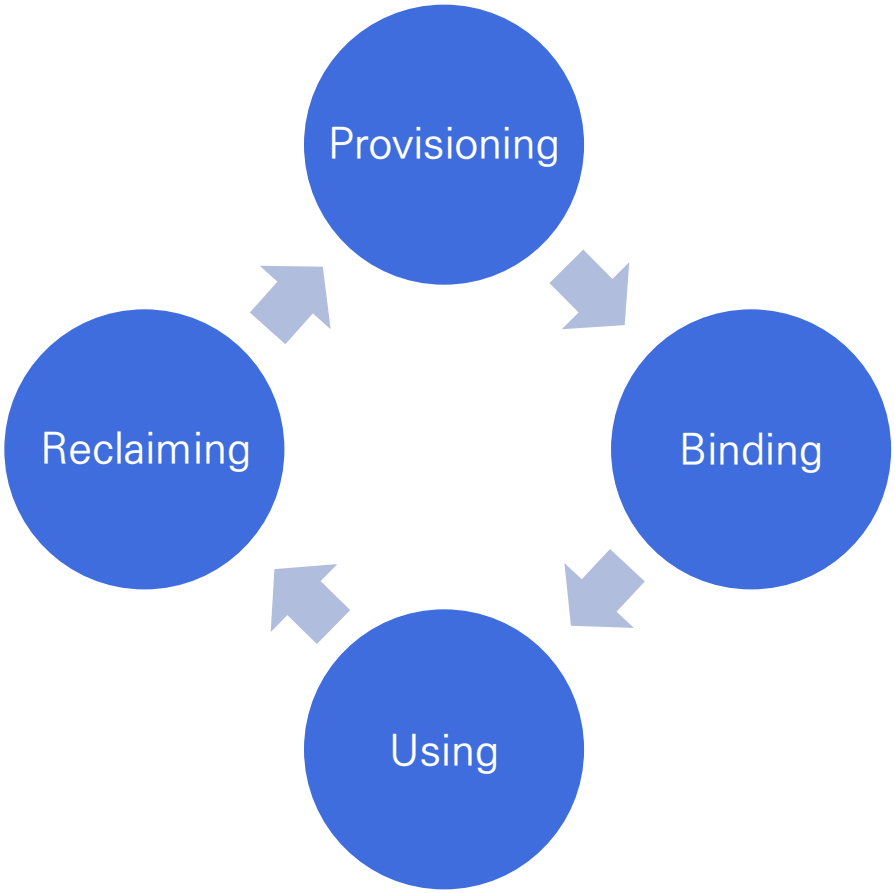
StorageClass

- PV로 확보한 스토리지 종류를 정의 하는 리소스
- PV의 동적 프로비저닝일 때 사용되며 동적 프로비저닝을 활성화 하려면 하나 이상의 스토리지 클래스가 미리 생성 되어 있어야함
- 스토리지 클래스를 미리 정의하면 그에 맞는 PV를 바로 생성할 수 있음

03 Kubernetes

PV, PVC 생명주기

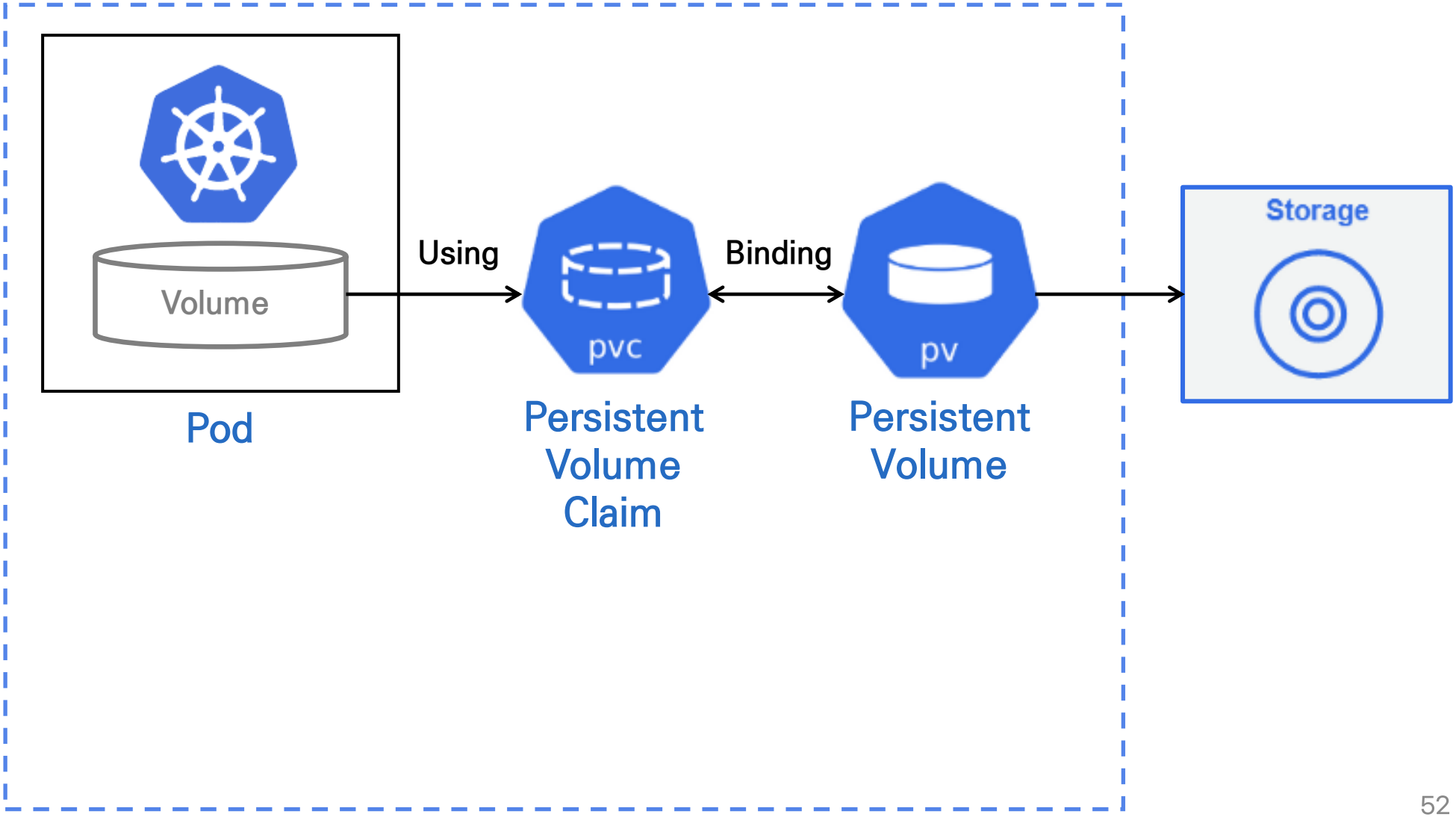
생명주기



- **Provisioning**
  정적 방법과 동적 방법 두 가지가 있으며 PV를 생성하는 단계
- **Binding**
  PVC가 원하는 접근방법, 스토리지 용량 등을 요청하면 그에 맞는 PV가 연결되는 단계
- **Using**
  파드가 유지되는 동안 파드에서 설정된 PVC가 계속해서 사용되고 있는 단계
- **Reclaiming**
  사용이 끝난 PVC는 삭제되고 사용 중이던 PV는 초기화 되는 과정

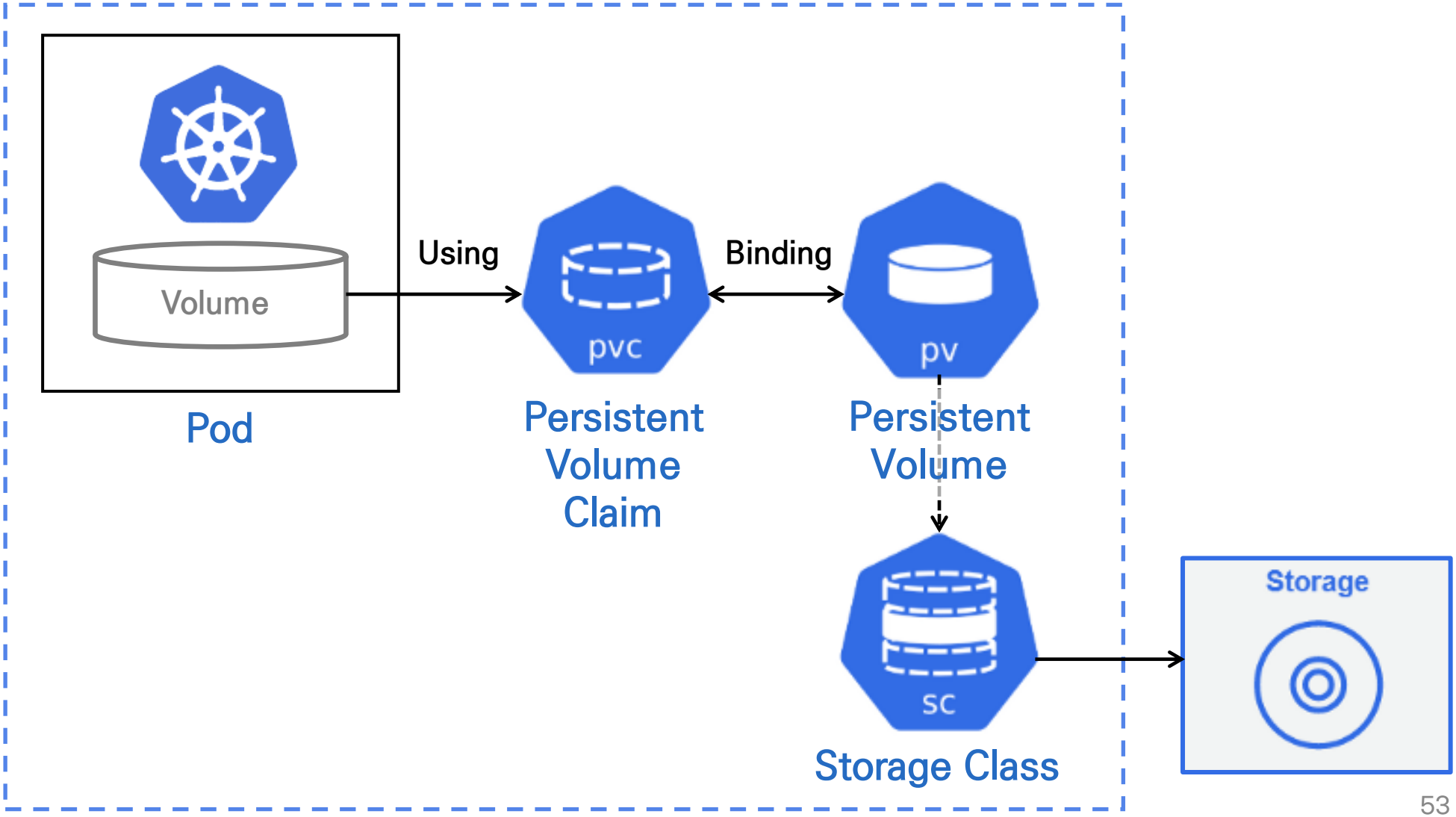
03 Kubernetes

- Volume 구조(정적 프로비저닝)



03 Kubernetes

- Volume 구조(동적 프로비저닝)



03 Kubernetes

- ConfigMap

컨피그맵 특징

ConfigMap

- 컨테이너와 분리해서 환경설정 제공
- 키-값 형태의 설정 정보를 저장하는 일종의 저장소 역할
- 간단한 문자부터 큰 설정 파일까지 값으로 가질 수 있음
- 동일한 파드라도 여러 개의 컨피그맵을 사용할 수 있음

03 Kubernetes

- Secret

시크릿 특징

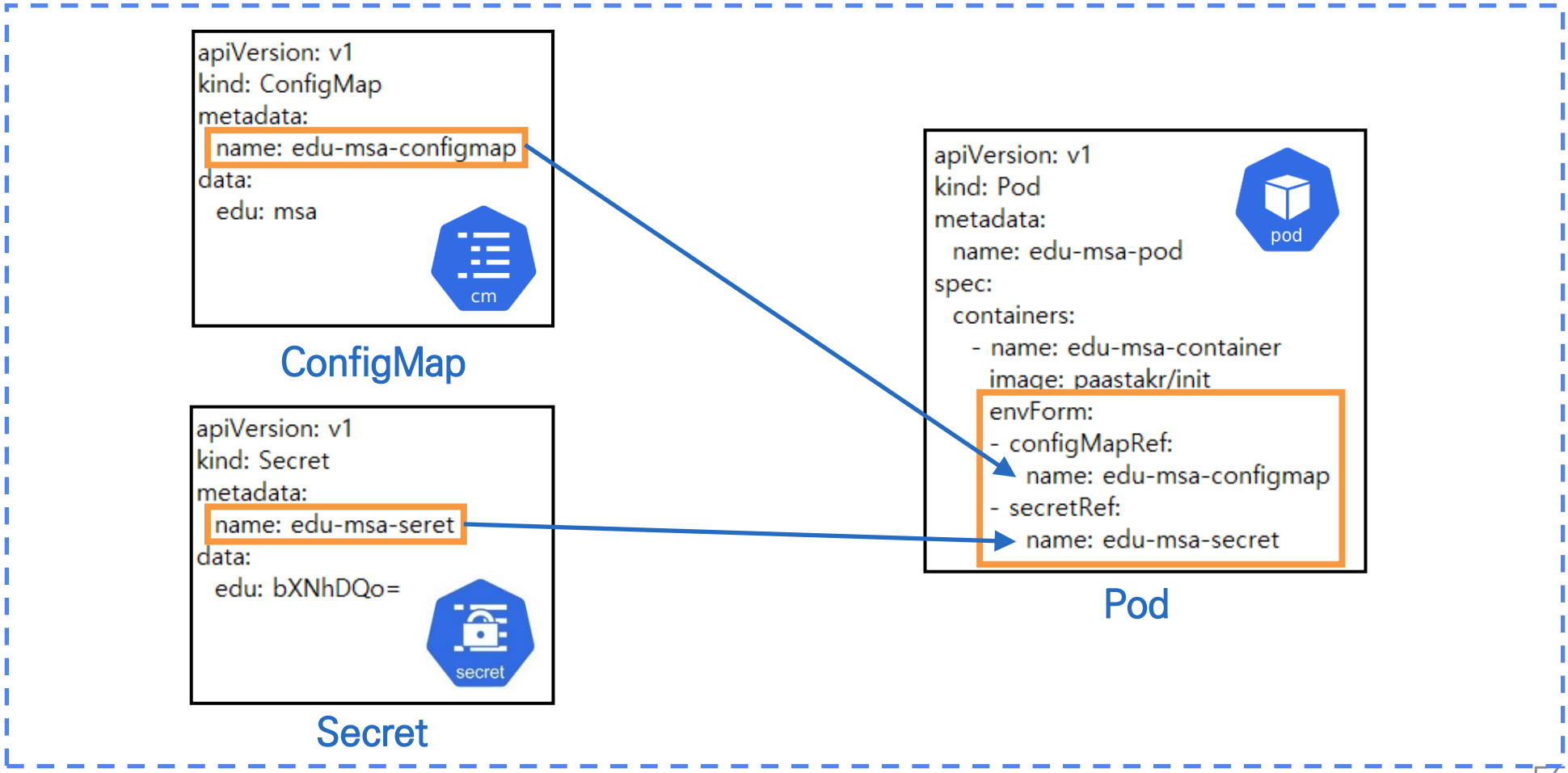
Secret

- 보안유지가 필요한 자격증명 및 개인 암호화 키 같은 정보를 저장 및 관리하는 역할
- Base64 인코딩으로 생성
- 키-값 형태의 설정 정보를 저장하는 일종의 저장소 역할
- 메모리에 저장됨
- 개별 시크릿의 최대 크기는 1MB까지 정의
- 모든 파드에는 자동으로 연결된 시크릿 볼륨이 존재

03 Kubernetes

- **ConfigMap, Secret 사용 방법 소개**

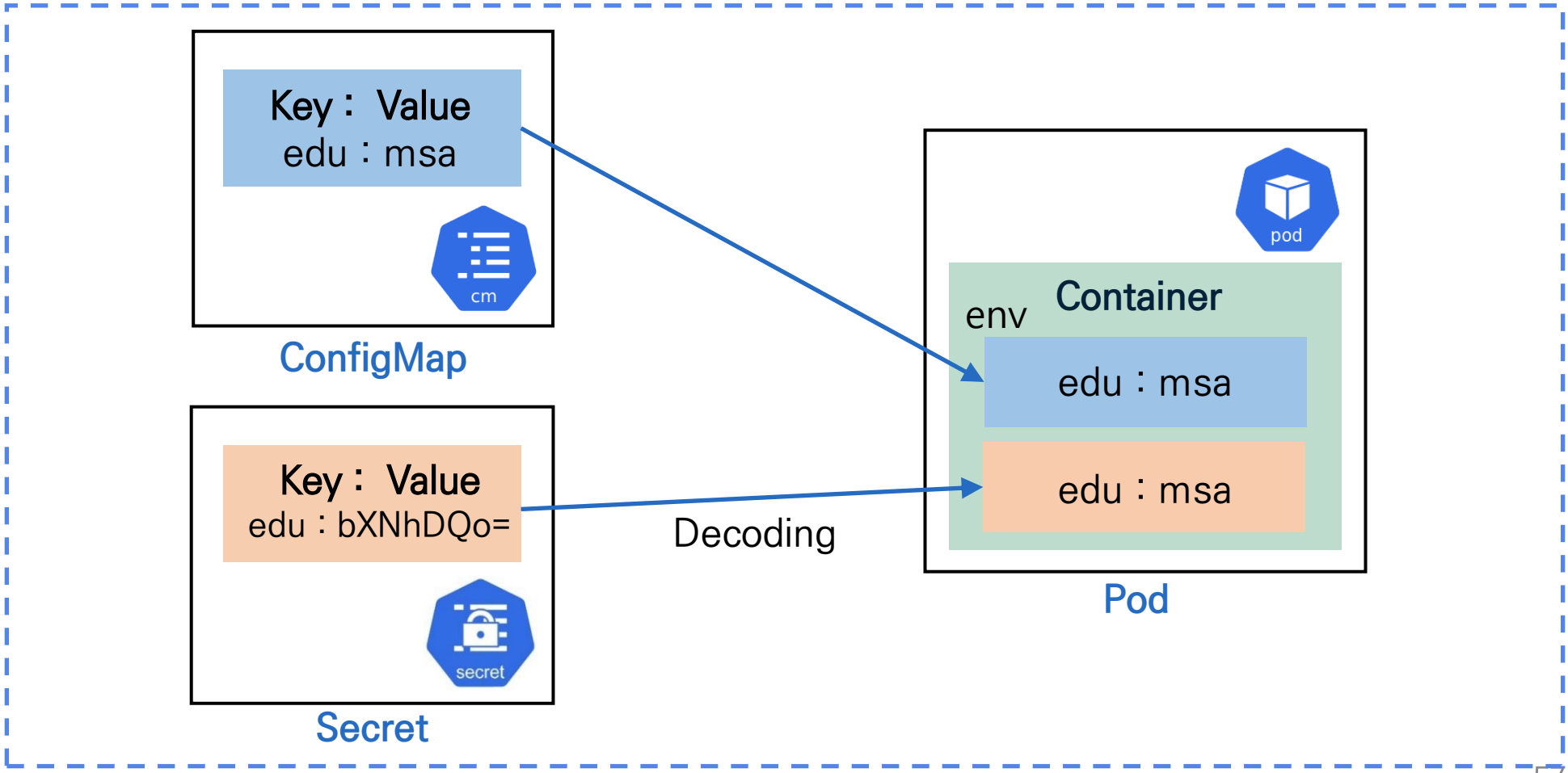
✔ 파드 생성시 컨피그맵과 시크릿 정보를 입력



03 Kubernetes

- **ConfigMap, Secret 사용 방법 소개**

✔ 파드의 컨테이너 환경 변수에 컨피그맵의 데이터와 디코딩 된 시크릿의 데이터가 들어 감



03 Kubernetes

Kubectl 이해

kubectl 개요

정의

쿠버네티스 API를 사용하여 쿠버네티스 클러스터의 컨트롤 플레인과 통신하기 위한 CLI

기본 형태

$ kubectl [command] [TYPE] [NAME] [flags]

요소 설명

- command: 하나 이상의 리소스에서 수행하려는 동작 (create, apply, get, describe, delete 등)
- TYPE: 리소스 타입 (all, pods, deployments, services 등)
  리소스 타입은 대소문자를 구분하지 않으며 단수형, 복수형 또는 약어 형식을 지정할 수 있다.
- NAME: 리소스 이름
  이름은 대소문자를 구분한다. 이름을 생략하면, 모든 리소스에 대한 세부 사항이 표시된다.
- flags: 선택적 플래그. (-f FILENAME, -n NAMESPACE -o wide 등)

출처 : Kubernetes 공식 문서(https://kubernetes.io/ko/docs/reference/kubectl/)

03 Kubernetes

Kubectl 이해

(생성) apply

명령 의미

파일이나 표준입력(stdin)으로부터 리소스에 구성 변경 사항을 적용

기본 형태

$ kubectl apply -f FILENAME [flags]

사용 예시

yaml 파일의 리소스들을 생성
(생성 시 yaml 파일이 있는 위치에서 실행)

```
I:₩kubeyaml>kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx created
```

(생성) create

명령 의미

파일이나 표준입력에서 하나 이상의 리소스를 생성

기본 형태

$ kubectl create -f FILENAME [flags]

사용 예시

```
I:₩kubeyaml>kubectl create -f nginx-deployment.yaml
deployment.apps/nginx created
```

출처 : Kubernetes 공식 문서(https://kubernetes.io/ko/docs/reference/kubectl/)

03 Kubernetes

Kubectl 이해

(조회) get

명령 의미

하나 이상의 리소스를 조회

기본 형태

```
$ kubectl get [TYPE] [flags]
$ kubectl get [TYPE] [NAME]
$ kubectl get [TYPE] [NAME] [flags]
```

사용 예시

```
$ kubectl get pods
```

```
I:₩kubeyaml>kubectl get pods
NAME                      READY   STATUS    RESTARTS   AGE
nginx-559d75d76b-4zhsz    1/1     Running   0          2m2s
nginx-559d75d76b-7b8s2    1/1     Running   0          2m2s
```

(상세조회) describe

명령 의미

하나 이상의 리소스의 상세한 상태 조회

기본 형태

```
$ kubectl describe -f FILENAME
```

사용 예시

```
$ kubectl describpe pod nginx-599d75d6b-4zhsz
```

```
I:₩kubeyaml>kubectl describe pod nginx-559d75d76b-4zhsz
Name:         nginx-559d75d76b-4zhsz
Namespace:    paas-83cc724a-4425-4f1e-aab4-63a4f23fcd99-caas
Priority:     0
Node:         playpark-caas-worker-6/10.0.50.254
Start Time:   Fri, 29 Oct 2021 14:47:49 +0900
Labels:       app=nginx
              pod-template-hash=559d75d76b
Annotations:  cni.projectcalico.org/podIP: 10.233.117.192/32
              cni.projectcalico.org/podIPs: 10.233.117.192/32
              kubernetes.io/limit-ranger: LimitRanger plugin set: cpu, memory request for container nginx; cpu, memory limit for container nginx
Status:       Running
IP:           10.233.117.192
IPs:
  IP:           10.233.117.192
Controlled By:  ReplicaSet/nginx-559d75d76b
Containers:
```


출처 : Kubernetes 공식 문서(https://kubernetes.io/ko/docs/reference/kubectl/)

03 Kubernetes

Kubectl 이해

(로그조회) logs

명령 의미

특정 이름을 가진 리소스의 로그 정보 출력

기본 형태

$ kubectl logs [flags] [NAME]

사용 예시

$ kubectl logs -f edu-msa-zuul-647b787d7-g5st7

(Editor 수정) edit

명령 의미

특정 이름을 가진 리소스 정보 수정

기본 형태

$ kubectl edit [TYPE] [NAME]

사용 예시

$ kubectl edit deployment nginx

출처 : Kubernetes 공식 문서(https://kubernetes.io/ko/docs/reference/kubectl/)

03 Kubernetes

Kubernetes 리소스 관리

쿠버네티스 리소스는 YAML 또는 JSON으로 작성된 구성 파일과 함께
kubectl 커맨드 라인 툴을 이용하여 관리

```yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
```

YAML

```json
{
    "kind": "Pod",
    "apiVersion": "v1",
    "metadata": {
        "name": "nginx",
        "labels": {
            "run": "nginx"
        }
    },
    "spec": {
        "containers": [
            {
                "name": "nginx",
                "image": "nginx"
            }
        ]
    }
}
```

JSON

```
[root@k8s-master-01 ~]# kubectl create -f nginx.yaml
pod/nginx created
[root@k8s-master-01 ~]# kubectl get pod nginx
NAME    READY   STATUS    RESTARTS   AGE
nginx   1/1     Running   0          10s
```

kubectl 커맨드

03 Kubernetes

- **yaml 파일 작성 방법**

디플로이먼트 예시 (1/7)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- apiVersion
  특정 리소스를 생성하기 위해 사용할 쿠버네티스 API 버전을 명세

- kind
  어떤 종류의 리소스를 생성하고자 하는지 생성할 리소스 타입을 명세

03 Kubernetes

- **yaml 파일 작성 방법**

디플로이먼트 예시 (2/7)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- metadata
  리소스에 이름을 부여하고 리소스를 유일하게 구분 짓는 데이터
- metadata.name
  리소스의 이름을 명세
- metadata.labels
  키-값 쌍으로 구성되어 특정 쿠버네티스 리소스만 나열 또는 검색할 때 사용
- metadata.labels.app
  리소스의 레이블을 설정

03 Kubernetes

- **yaml 파일 작성 방법**

디플로이먼트 예시 (3/7)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- spec
  생성하고자 하는 리소스에 대한 내용을 구체적으로 정의
- spec.replicas
  띄우고자 하는 파드의 개수를 설정
- spec.selector
  어떤 파드를 컨트롤하고 감시해야 하는지 정의
- spec.selector.matchLabels
  metadata.labels와 동일한 설정으로 맵핑하여 동일한 레이블을 가진 파드의 컨테이너를 카운팅하여 현재 상태를 측정

03 Kubernetes

- **yaml 파일 작성 방법**

디플로이먼트 예시 (4/7)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- spec.template
  어떤 파드를 실행할지에 대한 정보를 설정

- spec.template.metadata
  metadata.labels와 동일한 설정으로 맵핑하여 동일한 레이블을 가진 파드를 실행

03 Kubernetes

- **yaml 파일 작성 방법**

디플로이먼트 예시 (5/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
    - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- spec.template.spec
  실행시킬 컨테이너에 대한 설정
- spec.template.spec.containers[]
  실행시킬 컨테이너의 이름, 이미지, 포트 번호 등을 설정
- spec.template.spec.containers[].name
  컨테이너 이름을 지정

03 Kubernetes

- **yaml 파일 작성 방법**

디플로이먼트 예시 (6/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
    - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- spec.template.spec.containers[].image
  컨테이너로 배포할 이미지를 지정
- spec.template.spec.containers[].imagePullPolicy
  이미지 다운로드 정책을 지정("Always"는 이미지 여부에 상관없이 다운로드)
- spec.template.spec.containers[].ports[]
  컨테이너 포트번호를 지정

03 Kubernetes

- **yaml 파일 작성 방법**

디플로이먼트 예시 (7/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
    - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- spec.template.spec.imagePullSecret
  이미지 저장소에 접근하기 위한 인증정보
- spec.template.spec.nodeSelector
  파드를 배포할 특정 노드를 지정

03 Kubernetes

- **yaml 파일 작성 방법**

서비스 예시 (1/2)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
  - nodePort: ${EDU_MSA_BOARD}
    port: 28082
    protocol: TCP
    targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

- spec.ports.[].nodePort
  외부에서 접근 가능한 포트 번호를 설정
  (외부 포트)
- spec.ports.[].port
  컨테이너 포트 번호를 설정(내부 포트)
- spec.ports[].protocol
  프로토콜 방식을 설정
- spec.ports[].targetport
  접근하고자 하는 컨테이너 포트 번호를 설정

03 Kubernetes

- **yaml 파일 작성 방법**

서비스 예시 (2/2)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
  - nodePort: ${EDU_MSA_BOARD}
    port: 28082
    protocol: TCP
    targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

- spec.type
  외부에 노출하고자 하는 방식을 설정
  (“NodePort”는 포트 번호를 통해 외부에서 접근할 수 있는 방법)

03 Kubernetes

yaml 파일 작성 방법

yaml 파일 통합

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
    spec:
      containers:
      - name: board-msa
        image: paastakr/edu-msa-board:latest
        imagePullPolicy: Always
        ports:
        - containerPort: 28082
---
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
  - nodePort: 30201
    port: 28082
    protocol: TCP
    targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

Deployment

Service

- 서로 관련된 여러 yaml 파일을 하나의 파일로 작성하기 위해서는 중간에 “---” 로 구분 지어 작성

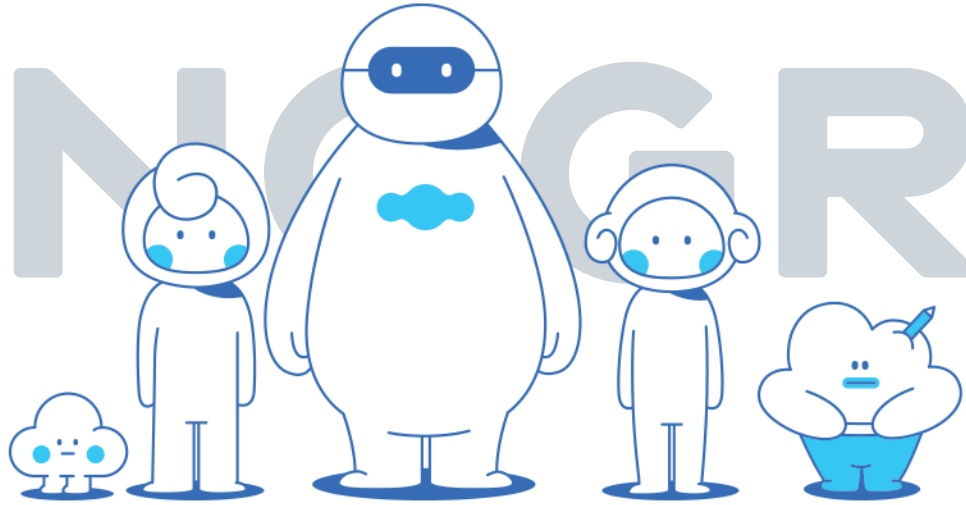
03 Kubernetes

Kubernetes 배포 도구

kubeadm	일반적인 서버 클러스터 환경에서도 쿠버네티스를 쉽게 설치할 수 있게 해주는 관리 툴로서 클러스터를 빠르고 일관되게 설정할 수 있음
kops	자동화된 프로비저닝 시스템으로 AWS에 쉽고 빠르게 쿠버네티스 클러스터를 설치 가능
Kubespray	Ansible을 통해 쿠버네티스 클러스터를 유연하고 쉽게 배포 및 관리할 수 있는 강력한 오픈 소스 툴 * GCE, Azure, OpenStack, AWS, vSphere, Equinix Metal(전 Packet), Oracle Cloud 등 여러 플랫폼에서 쿠버네티스 클러스터 설치 가능

출처 : 쿠버네티스 홈페이지

수고하셨습니다.



INNOGRID